

EXITAI · VERIFIED CODE WALKTHROUGH

Code Line by Line Rewritten

How the current code starts, routes requests, advances an exit case, runs agents, retrieves policy context, and persists state.

Source material: docs/ExitAI_Code_LineByLine.pdf
Verified against repository files on 22 September 2026

READING GUIDE

Scope and notation

This rewrite follows the source PDF but resolves every path against the current repository. It explains code behavior only. It deliberately omits project planning, deployment guidance, business justification, and generic AI theory.

Two execution models exist. The live browser does *not* invoke `agents/hub/supervisor.py`. Browser actions call Supabase directly, call three Edge Functions, and use `agents/service.py` for Python-only work. The supervisor is used by the CLI and test/capstone runners.

1. Actual code structure
2. How the code starts
3. Complete browser lifecycle
4. Agent structure and live status
5. Supervisor / hub state machine
6. Service and API code
7. RAG code path
8. Database flow
9. Line-by-line source walkthrough

SECTION 1

Actual code structure

project/	
├── agents/	
│ ├── core/	config, LLM gateway, prompts, tracing, email, calendar
│ ├── hub/	
│ │ ├── supervisor.py	CLI LangGraph pipeline
│ │ └── e2e_automation.py	standalone capstone graph
│ ├── spokes/	operational agents and thin compatibility wrappers
│ ├── analytics/	manually-run aggregate agents
│ ├── service.py	local browser-to-Python bridge on port 8787
│ └── run_case.py	CLI wrapper around supervisor.run_case
├── src/	
│ ├── main.jsx	React mount
│ ├── App.jsx	auth and role lookup
│ ├── routes/	employee, manager, IT, finance, HR pages/layouts
│ ├── components/	shared UI
│ ├── lib/	Supabase client and derived-state helpers
│ └── styles/	UI styling only
├── supabase/	
│ ├── functions/	
│ │ ├── submit-resignation/index.ts	
│ │ ├── ask/index.ts	
│ │ └── forward-to-hr/index.ts	
│ └── migrations/	schema, RLS, views, RPCs, later columns
├── scripts/	
│ ├── exit_policy.md	RAG source document
│ ├── ingest_docs.js	RAG ingestion
│ ├── seed/	demo data
│ └── verify/	browser/DB/agent verification scripts
├── prompts/	system prompts loaded by Python agents
└── docs/	documentation and PDF renderers

Excluded from detailed treatment: empty `__init__.py` files, CSS, generated PDFs, images, one-time OAuth setup, seed data, and verification scripts. They do not define the runtime flow explained here.

Frontend and API files

src/main.jsx	
PURPOSE	Mounts React and imports global CSS.
CALLED BY	The browser loads index.html.
CALLS	<code>createRoot(...).render()</code> .
OUTPUT	A mounted React component tree.

src/App.jsx	
PURPOSE	Resolves the Supabase session and <code>profiles.role</code> , then chooses the authenticated dashboard.
CALLED BY	<code>src/main.jsx</code> .
CALLS	<code>supabase.auth</code> , <code>profiles</code> , <code>AppRoutes</code> .
OUTPUT	<code>LoginPage</code> or role-scoped routes.

src/routes/AppRoutes.jsx

PURPOSE	Maps URLs to role-guarded layouts/pages.
CALLED BY	App.
CALLS	Employee/Manager/IT/Finance/HR route modules.
OUTPUT	The selected page or a redirect.

src/routes/employee/EmployeePages.jsx [Live browser](#)

PURPOSE	Implements resignation, employee tasks, policy Q&A, uploads, and exit interview submission.
CALLED BY	Employee routes.
CALLS	submit-resignation, ask, forward-to-hr, Supabase tables/storage, local service.
OUTPUT	Database writes plus rendered employee state.

src/routes/manager/ManagerPages.jsx [Live browser](#)

PURPOSE	Approves or rejects manager-stage KT tasks and triggers the manager gate handoff.
CALLED BY	Manager routes.
CALLS	exit_tasks and /manager-approve or /reject-manager-task.
OUTPUT	Task status, escalation, or IT-stage advancement.

src/routes/it/ItPages.jsx [Live browser](#)

PURPOSE	Approves IT tasks and triggers execution/verification.
CALLED BY	IT routes.
CALLS	exit_tasks and /execute-deprovisioning.
OUTPUT	Done task plus agent_runs verification audit.

src/routes/finance/FinancePages.jsx [Live browser](#)

PURPOSE	Settles or rejects dues.
CALLED BY	Finance routes.
CALLS	finance RPCs and /finance-settle-check.
OUTPUT	finance_cleared/dues_note and recalculated finance/compliance/risk.

src/routes/hr/HrPages.jsx [Live browser](#)

PURPOSE	Handles escalation transitions and final relieving-letter issue.
CALLED BY	HR routes.
CALLS	exit_tasks, exit_cases, /escalation-audit, /issue-relieving-letter.
OUTPUT	Resolved/rerouted escalation or completed case.

supabase/functions/submit-resignation/index.ts [Live browser](#)

PURPOSE	Authenticates the employee and creates one exit_cases row.
CALLED BY	EmployeePages.Resignation.handleSubmit.
CALLS	Supabase Auth, profiles, exit_cases.
OUTPUT	{ ok, case } JSON.

supabase/functions/ask/index.ts

Live browser

PURPOSE	Embeds a question, retrieves policy chunks, asks the LLM, and builds citations.
CALLED BY	EmployeePages.Dashboard.handleAsk.
CALLS	embedding gateway, match_exit_docs RPC, message gateway.
OUTPUT	{ answer, sources, refused, general } JSON.

supabase/functions/forward-to-hr/index.ts

Live browser

PURPOSE	Emails an explicitly forwarded unanswered question; logs failed delivery.
CALLED BY	EmployeePages.Dashboard.handleForward.
CALLS	Supabase Auth/profiles, SMTP, exit_cases, agent_runs.
OUTPUT	Immediate or delayed-success JSON.

Python runtime files

agents/service.py

Live browser

PURPOSE	Exposes nine localhost POST routes for browser-triggered Python work.
CALLED BY	Browser fetch calls.
CALLS	Operational agents, db, trace logger.
OUTPUT	JSON responses and agent/database side effects.

agents/hub/supervisor.py

CLI/test, not live browser

PURPOSE	Builds the ordered LangGraph used for a complete simulated case run.
CALLED BY	run_case.py, e2e_automation.py, tests, direct module execution.
CALLS	Checklist/KT/IT/compliance/finance/risk/routing agents.
OUTPUT	Final SupervisorState and persisted agent_runs/tasks.

agents/run_case.py

CLI only

PURPOSE	Parses CLI arguments and invokes supervisor.run_case.
CALLED BY	python -m agents.run_case.
CALLS	agents.hub.supervisor.run_case.
OUTPUT	Trace output and final log.

agents/hub/e2e_automation.py

Standalone; not live browser

PURPOSE	Composes initiation, supervisor, per-case SLA scan, and finalization.
CALLED BY	Direct module execution/tests.
CALLS	service.activate_case, supervisor.run_case, sla_escalation.
OUTPUT	Completed/blocked outcome and audit row.

agents/core/config.py

PURPOSE	Loads environment values and constructs the shared service-role Supabase client.
CALLED BY	All Python modules that import db/config constants.
CALLS	dotenv, create_client.
OUTPUT	Module constants and db client.

agents/core/llm.py

PURPOSE	Makes every Python LLM request and optionally parses JSON.
CALLED BY	LLM-backed spokes and analytics agents.
CALLS	Portkey/Anthropic-compatible /v1/messages.
OUTPUT	Text or dict.

agents/core/prompts.py

PURPOSE	Reads prompt markdown from prompts/.
CALLED BY	LLM-backed agents at import time.
CALLS	Path.read_text.
OUTPUT	Prompt string.

agents/core/trace.py

PURPOSE	Wraps graph nodes and logs LLM/DB timing.
CALLED BY	@traced_node and explicit log_llm/log_db calls.
CALLS	stdout only.
OUTPUT	Trace lines; no workflow state change.

agents/core/notifications.py

PURPOSE	Builds and sends/logs email templates.
CALLED BY	email_drafting_agent and manual overdue scan.
CALLS	profiles, exit_tasks, exit_cases, Gmail SMTP.
OUTPUT	Per-recipient delivery dictionaries.

agents/core/calendar_booking.py

PURPOSE	Creates KT calendar events when configured.
CALLED BY	hr_agent._persist_checklist.
CALLS	Google Calendar API, exit_tasks.
OUTPUT	Event result and kt_event_id update.

Data and RAG files

scripts/ingest_docs.js	
PURPOSE	Chunks scripts/exit_policy.md, embeds chunks, replaces rows for that source.
CALLED BY	Manual node scripts/ingest_docs.js.
CALLS	Portkey embeddings, exit_docs.
OUTPUT	Fresh vector rows.

supabase/migrations/0001_schema.sql	
PURPOSE	Creates core tables and pgvector index.
CALLED BY	Supabase migration runner.
CALLS	PostgreSQL/pgvector.
OUTPUT	profiles, exit_cases, exit_tasks, exit_interviews, trend_alerts, exit_docs.

supabase/migrations/0002_rls.sql	
PURPOSE	Enables RLS and defines role-specific policies/views.
CALLED BY	Supabase migration runner.
CALLS	auth.uid(), profiles role helper.
OUTPUT	Database-enforced row access.

supabase/migrations/0003_rag.sql	
PURPOSE	Defines cosine-similarity RPC for policy chunks.
CALLED BY	ask Edge Function through db.rpc.
CALLS	exit_docs and pgvector <=>.
OUTPUT	Top matching chunks with similarity.

SECTION 2

How the code starts

Browser startup

```
index.html #root
↓ src/main.jsx · createRoot(...).render(<App />)
↓ src/App.jsx · App()
↓ supabase.auth.getSession() + profiles.role
↓ src/routes/AppRoutes.jsx · AppRoutes({ role, session })
↓ role layout loads rows through RLS-scoped Supabase client
↓ page action invokes Edge Function, RPC, direct table update, or localhost:8787
↓ layout reload() reads fresh rows and React re-renders
```

`src/lib/supabase.js` supplies one anon-key client. Browser identity comes from its session JWT. Database RLS—not the JSX route guard—is the data boundary.

Browser resignation entry

```
src/routes/employee/EmployeePages.jsx · Resignation.handleSubmit
↓ supabase.functions.invoke('submit-resignation')
↓ supabase/functions/submit-resignation/index.ts · Deno.serve handler
↓ Auth getUser → profiles lookup → exit_cases insert/return existing
↓ browser POST http://localhost:8787/activate-exit
↓ agents/service.py · Handler.do_POST → activate_case
↓ hr_agent.generate_checklist → exit_tasks(hr, manager)
↓ email_drafting_agent.resignation_notice → notifications
↓ exit_cases.status = in_progress
↓ JSON response; browser navigates to /employee
```

CLI entry

```
python -m agents.run_case CASE_ID [--reject] [--kt-text FILE] [--interview-text FILE]
↓ agents/run_case.py · _read() and __main__ block
↓ agents/hub/supervisor.py · run_case(...)
↓ supervisor_graph.invoke(initial SupervisorState)
↓ hr → manager_gate → approved: it → compliance → finance → assess
    rejected: escalate → END
↓ final SupervisorState printed with elapsed time
```

The CLI gate is simulated. `simulate_rejection` selects the branch. It does not wait for a manager's browser action or inspect manager task status.

Standalone capstone entry

```
python -m agents.hub.e2e_automation CASE_ID [--reject]
  ↓ _initiate → service.activate_case
  ↓ _coordinate → supervisor.run_case
  ↓ _sla_check → sla_escalation.find_breaches + _escalate
  ↓ _finalize → read compliance/finance task rows
  ↓ exit_cases.status completed only when both rows are done
```

This is not the browser route. It is an explicit all-in-one runner.

SECTION 3

Complete live browser lifecycle

Employee resignation

- ↓ HR checklist generation / employee completion
- ↓ Manager KT approval — rejection → HR escalation
- ↓ IT plan and human IT approval
- ↓ execute / verify / audit + compliance re-check
- ↓ Finance dues decision
- ↓ finance task + compliance + risk re-check
- ↓ HR relieving-letter gate
- ↓ `exit_cases.status = completed`

Transition	File / function	Input	What it does	Reads / writes	Next
Resign	<code>EmployeePages.js</code> <code>Resignation.handleSubmit</code>	last working day, reason; session JWT	Calls Edge Function, then local activation service.	Edge Function writes <code>exit_cases</code> .	Employee dashboard.
Activate	<code>agents/service.py</code> <code>activate_case</code>	<code>case_id</code>	Generates HR/manager tasks, marks case in progress, sends manager notice.	Reads/writes <code>exit_cases</code> , writes <code>exit_tasks / agent_runs</code> .	Employee completes HR tasks; manager reviews KT.
Employee task done	<code>EmployeePages.js</code> <code>useMarkDone.markDone</code>	task id	Direct status=done update; RLS restricts to employee's HR- stage tasks.	Writes <code>exit_tasks</code> .	Manager gate remains separate.
Manager approval	<code>ManagerPages.jsx</code> <code>useApprove /</code> <code>useSignClearance</code>	manager task/case id	Marks a KT task done, then service verifies all KT rows and creates IT plan.	Writes <code>exit_tasks</code> , <code>agent_runs</code> ; reads case/tasks.	IT dashboard receives stage=it tasks.
Manager rejection	<code>ManagerPages.jsx</code> → <code>service.reject_manager_task</code>	<code>case_id</code> , <code>task_id</code> , reason	Creates/reopens one manager escalation and logs it.	Writes <code>exit_tasks</code> , <code>agent_runs</code> .	Stops before IT until HR action.
HR escalation action	<code>HrPages.jsx</code> <code>useEscalationAction.act</code>	task, rerouted/resolved, actor	RLS update changes <code>escalation_state</code> ; service appends audit and closes resolved task.	Writes <code>exit_tasks</code> , <code>agent_runs</code> .	Rerouted returns to manager; resolved removes blocker.
IT approval	<code>ItPages.jsx</code> <code>useApprove.approveTask</code>	IT task id and case id	Marks task done; runs <code>MockITAdapter</code> <code>execute/verify/audit</code> ; reruns compliance.	Writes <code>exit_tasks</code> , <code>agent_runs</code> , compliance rows.	Finance can proceed when prior tasks are done.

Transition	File / function	Input	What it does	Reads / writes	Next
Document upload	<code>EmployeePages.jsx</code> <code>Documents.handleUpload</code>	file and doc_type	Uploads to bucket, inserts metadata, triggers OCR validation and compliance.	Storage <code>exit-documents</code> ; <code>case_documents</code> , <code>compliance rows</code> .	Validated NDA/asset form can satisfy compliance items.
Exit interview	<code>EmployeePages.jsx</code> <code>ExitInterview.handleSubmit</code>	reason, feedback, recommend, comments	Inserts raw answers, then LLM adds summary/sentiment/theses/rehire fields.	Writes <code>exit_interviews</code> .	Risk reads sentiment later.
Finance settle	<code>FinancePages.jsx</code> <code>useSettle.settle</code>	case_id, dues note	RPC sets finance flag; service recomputes finance, compliance, and risk.	Writes <code>exit_cases</code> , <code>exit_tasks</code> , <code>compliance_checks</code> , <code>agent_runs</code> .	HR final gate.
Finance reject	<code>FinancePages.jsx</code> <code>useReject.reject</code>	case_id, reason	RPC sets <code>finance_cleared=false</code> and records the reason.	Writes <code>exit_cases</code> .	Finance remains held; no Python endpoint runs.
Relieving letter	<code>HrPages.jsx</code> <code>useIssueRelievingLetter.issue</code>	case_id, HR user id	Direct update succeeds only through database gate; service sends notice.	Writes <code>exit_cases</code> ; email attempt in <code>agent_runs</code> .	Case is completed.

Compliance is event-driven in the browser path. It is rerun after manager advancement, document validation, IT execution, and finance settlement. It is not a free-running background process.

SECTION 4

Agent structure

The “agent” label covers LangGraph subgraphs, deterministic evaluators, wrappers, and LLM-backed functions. The table below states exactly which kind each one is and whether the browser reaches it.

Supervisor / Orchestrator (#1)

Not part of the live browser execution path

Field	Actual implementation
File	agents/hub/supervisor.py
Purpose	LangGraph routes HR→manager gate and branches to IT or escalation, then compliance→finance→assess.
Main function	run_case(case_id, ...)
Called by	agents/run_case.py; e2e_automation.py; tests; direct module execution
Calls	LLM: Only through delegated HR/IT/interview agents. Database: Activates exit_cases; delegated agents write tasks/checks/risk; _record inserts agent_runs.
Input	case_id, optional KT/interview text, simulate_rejection
Processing	LangGraph routes HR→manager gate and branches to IT or escalation, then compliance→finance→assess.
LLM call	Only through delegated HR/IT/interview agents.
Database operations	Activates exit_cases; delegated agents write tasks/checks/risk; _record inserts agent_runs.
Output	Final SupervisorState with log.
Next step	Caller receives final state; no browser response uses it.

Checklist Generator (#5) [Live browser](#)

Field	Actual implementation
File	<code>agents/spokes/checklist_generator_agent.py</code>
Purpose	Thin traced wrapper over <code>hr_agent.generate_checklist</code> .
Main function	<code>generate(case_id, force=False)</code>
Called by	<code>service.activate_case</code> ; <code>supervisor._hr_stage</code>
Calls	LLM: Indirectly: <code>hr_agent</code> calls <code>ask_claude_json</code> . Database: <code>hr_agent</code> reads <code>exit_cases</code> and inserts <code>exit_tasks</code> .
Input	<code>case_id</code> , optional <code>force</code>
Processing	Thin traced wrapper over <code>hr_agent.generate_checklist</code> .
LLM call	Indirectly: <code>hr_agent</code> calls <code>ask_claude_json</code> .
Database operations	<code>hr_agent</code> reads <code>exit_cases</code> and inserts <code>exit_tasks</code> .
Output	Checklist graph result or skipped flag.
Next step	Employee/manager work the generated tasks.

HR checklist / KT review (#2) [Live browser for checklist; KT review CLI only](#)

Field	Actual implementation
File	<code>agents/spokes/hr_agent.py</code>
Purpose	Two separate LangGraphs: <code>generate→persist</code> and <code>review→persist</code> .
Main function	<code>generate_checklist</code> ; <code>review_kt_document</code>
Called by	Checklist wrapper; KT wrapper
Calls	LLM: <code>ask_claude_json</code> with <code>hr/checklist_system.md</code> or <code>hr/kt_review_system.md</code> . Database: Reads <code>exit_cases/tasks</code> ; writes <code>exit_tasks</code> and <code>kt_reviews</code> ; may update <code>kt_event_id</code> via calendar.
Input	case row or KT text
Processing	Two separate LangGraphs: <code>generate→persist</code> and <code>review→persist</code> .
LLM call	<code>ask_claude_json</code> with <code>hr/checklist_system.md</code> or <code>hr/kt_review_system.md</code> .
Database operations	Reads <code>exit_cases/tasks</code> ; writes <code>exit_tasks</code> and <code>kt_reviews</code> ; may update <code>kt_event_id</code> via calendar.
Output	Graph state containing result.
Next step	Manager tasks form the human gate; KT gaps create neutral follow-up tasks.

KT Document Reviewer (#10) Not part of the live browser execution path

Field	Actual implementation
File	<code>agents/spokes/kt_document_reviewer_agent.py</code>
Purpose	Thin traced wrapper over <code>hr_agent.review_kt_document</code> .
Main function	<code>review(case_id, kt_text)</code>
Called by	<code>supervisor_hr_stage</code>
Calls	LLM: Indirect LLM call in <code>hr_agent</code> . Database: Writes <code>kt_reviews</code> and possible manager tasks through <code>hr_agent</code> .
Input	<code>case_id</code> , raw KT text
Processing	Thin traced wrapper over <code>hr_agent.review_kt_document</code> .
LLM call	Indirect LLM call in <code>hr_agent</code> .
Database operations	Writes <code>kt_reviews</code> and possible manager tasks through <code>hr_agent</code> .
Output	KT review graph result.
Next step	Supervisor proceeds to manager gate.

IT plan (#3) Live browser

Field	Actual implementation
File	<code>agents/spokes/it_agent.py</code>
Purpose	Fetches case, generates task titles, inserts stage=it rows; skips existing rows.
Main function	<code>generate_plan(case_id, force=False)</code>
Called by	<code>it_deprovisioning_agent.generate</code>
Calls	LLM: <code>ask_claude_json</code> with <code>it/deprovisioning_system.md</code> . Database: Reads <code>exit_cases/exit_tasks</code> ; inserts <code>exit_tasks</code> .
Input	<code>case_id</code>
Processing	Fetches case, generates task titles, inserts stage=it rows; skips existing rows.
LLM call	<code>ask_claude_json</code> with <code>it/deprovisioning_system.md</code> .
Database operations	Reads <code>exit_cases/exit_tasks</code> ; inserts <code>exit_tasks</code> .
Output	Graph state or skipped flag.
Next step	IT user approves tasks.

Automated IT Deprovisioning (#18)

[Live browser](#)

Field	Actual implementation
File	<code>agents/spokes/it_deprovisioning_agent.py</code>
Purpose	generate delegates to IT plan; execution classifies done tasks, calls MockITAdapter, independently verifies, and audits.
Main function	<code>generate; execute_approved_tasks</code>
Called by	<code>service.manager_approve; service.execute_deprovisioning; supervisor._it_stage</code>
Calls	LLM: None. Database: Reads <code>exit_tasks/agent_runs</code> ; inserts <code>agent_runs</code> .
Input	<code>case_id</code>
Processing	generate delegates to IT plan; execution classifies done tasks, calls MockITAdapter, independently verifies, and audits.
LLM call	None.
Database operations	Reads <code>exit_tasks/agent_runs</code> ; inserts <code>agent_runs</code> .
Output	Plan dict or list of execution records.
Next step	Compliance is rerun by service.

Compliance Verification (#13)

[Live browser](#)

Field	Actual implementation
File	<code>agents/spokes/compliance_agent.py</code>
Purpose	check→persist→item-level graph; matches tasks/documents and explicit manager/finance signals.
Main function	<code>run_for_case(case_id)</code>
Called by	<code>service event endpoints; supervisor._compliance_stage</code>
Calls	LLM: None. Database: Reads <code>tasks</code> , validated docs, manager <code>agent_runs</code> , <code>exit_cases.finance_cleared</code> ; updates/inserts <code>exit_tasks</code> ; upserts <code>compliance_checks</code> .
Input	<code>case_id</code>
Processing	check→persist→item-level graph; matches tasks/documents and explicit manager/finance signals.
LLM call	None.
Database operations	Reads <code>tasks</code> , validated docs, manager <code>agent_runs</code> , <code>exit_cases.finance_cleared</code> ; updates/inserts <code>exit_tasks</code> ; upserts <code>compliance_checks</code> .
Output	<code>{ result, items }</code> graph state.
Next step	Browser waits for later events; CLI continues to finance.

Finance (#4) [Live browser](#)

Field	Actual implementation
File	<code>agents/spokes/finance_agent.py</code>
Purpose	Requires at least one prior-stage task, all prior tasks done, and <code>finance_cleared=true</code> .
Main function	<code>check_clearance(case_id)</code>
Called by	<code>service.finance_settle_check</code> ; <code>supervisor._finance_stage</code>
Calls	LLM: None. Database: Reads <code>exit_tasks/exit_cases</code> ; inserts or updates finance task; may send completion email.
Input	<code>case_id</code>
Processing	Requires at least one prior-stage task, all prior tasks done, and <code>finance_cleared=true</code> .
LLM call	None.
Database operations	Reads <code>exit_tasks/exit_cases</code> ; inserts or updates finance task; may send completion email.
Output	Finance graph result.
Next step	Risk runs next in browser service; assess runs next in CLI.

Compliance & Risk (#12) [Live browser](#)

Field	Actual implementation
File	<code>agents/spokes/risk_agent.py</code>
Purpose	Scores tenure proxy, department criticality, interview sentiment, and incomplete tasks.
Main function	<code>run_for_case(case_id)</code>
Called by	<code>service.finance_settle_check</code> ; <code>supervisor._assess_stage</code>
Calls	LLM: None. Database: Reads <code>exit_cases</code> , <code>profiles</code> , <code>exit_interviews</code> , <code>exit_tasks</code> ; updates <code>exit_cases</code> ; inserts <code>agent_runs</code> .
Input	<code>case_id</code>
Processing	Scores tenure proxy, department criticality, interview sentiment, and incomplete tasks.
LLM call	None.
Database operations	Reads <code>exit_cases</code> , <code>profiles</code> , <code>exit_interviews</code> , <code>exit_tasks</code> ; updates <code>exit_cases</code> ; inserts <code>agent_runs</code> .
Output	<code>risk_score</code> , <code>risk_level</code> , <code>rehire_eligible</code> .
Next step	HR reads assessment fields.

Exit-Interview Intelligence (#8/#19)

Live browser per-case; longitudinal manual only

Field	Actual implementation
File	<code>agents/spokes/exit_intel_agent.py</code>
Purpose	Per-case analyze→persist; longitudinal counts repeated themes→trend alerts.
Main function	<code>run_per_case; run_longitudinal</code>
Called by	<code>service.submit_exit_interview</code> ; supervisor assess; manual module
Calls	LLM: <code>ask_claude_json</code> for per-case only. Database: Reads/writes <code>exit_interviews</code> ; longitudinal reads <code>exit_cases</code> and inserts <code>trend_alerts</code> .
Input	<code>case_id</code> + transcript, or all interviews
Processing	Per-case analyze→persist; longitudinal counts repeated themes→trend alerts.
LLM call	<code>ask_claude_json</code> for per-case only.
Database operations	Reads/writes <code>exit_interviews</code> ; longitudinal reads <code>exit_cases</code> and inserts <code>trend_alerts</code> .
Output	Analysis state or rising themes.
Next step	Risk can consume sentiment; HR reads interviews/trends.

Document Collection (#16)Live browser via `validate_one`

Field	Actual implementation
File	<code>agents/spokes/doc_collection.py</code>
Purpose	Downloads Storage object, OCRs it, applies document-specific keywords/name/date rules.
Main function	<code>validate_one; run</code>
Called by	<code>service.validate_document</code> ; manual module/tests
Calls	LLM: None. Database: Reads <code>exit_cases/case_documents</code> and Storage; updates <code>case_documents</code> ; inserts <code>agent_runs</code> .
Input	<code>case_id</code> , <code>document_id</code> or case batch
Processing	Downloads Storage object, OCRs it, applies document-specific keywords/name/date rules.
LLM call	None.
Database operations	Reads <code>exit_cases/case_documents</code> and Storage; updates <code>case_documents</code> ; inserts <code>agent_runs</code> .
Output	Validation dict or batch graph state.
Next step	Service reruns compliance.

Smart Routing (#11) Not part of the live browser execution path

Field	Actual implementation
File	<code>agents/spokes/smart_routing.py</code>
Purpose	Prefers department match, skips out-of-office candidates, records choice.
Main function	<code>pick_approver(case_id, stage)</code>
Called by	supervisor HR/manager/IT nodes
Calls	LLM: None. Database: Reads <code>exit_cases/profiles</code> ; writes <code>agent_runs</code> .
Input	<code>case_id</code> and <code>hr manager it</code>
Processing	Prefers department match, skips out-of-office candidates, records choice.
LLM call	None.
Database operations	Reads <code>exit_cases/profiles</code> ; writes <code>agent_runs</code> .
Output	Decision dict.
Next step	Supervisor continues its edge.

Multi-System Clearance (#15) Not part of the live browser execution path

Field	Actual implementation
File	<code>agents/spokes/multi_system_clearance.py</code>
Purpose	Queries HRMS/ITAM/Finance stand-in adapters with timeout/retry, then consolidates status.
Main function	<code>consolidate(case_id)</code>
Called by	<code>supervisor_compliance_stage</code>
Calls	LLM: None. Database: Reads <code>tasks/documents</code> ; writes <code>agent_runs</code> .
Input	<code>case_id</code>
Processing	Queries HRMS/ITAM/Finance stand-in adapters with timeout/retry, then consolidates status.
LLM call	None.
Database operations	Reads <code>tasks/documents</code> ; writes <code>agent_runs</code> .
Output	Per-system and overall status.
Next step	Supervisor proceeds to finance.

Email Drafting (#6)

[Live browser](#)

Field	Actual implementation
File	<code>agents/spokes/email_drafting_agent.py</code>
Purpose	Calls notifications function, catches errors, normalizes outcomes, audits attempt.
Main function	<code>template-specific wrapper functions</code>
Called by	service, finance, SLA, doc collection
Calls	LLM: None. Database: Writes agent_runs; notifications may read profiles.
Input	case/task/breach/template data
Processing	Calls notifications function, catches errors, normalizes outcomes, audits attempt.
LLM call	None.
Database operations	Writes agent_runs; notifications may read profiles.
Output	Delivery result without crashing caller.
Next step	Caller continues regardless of SMTP outcome.

Intelligent Rehire Assessment (#21)

Not part of the live browser execution path

Field	Actual implementation
File	<code>agents/spokes/rehire_agent.py</code>
Purpose	Reads already-computed case eligibility/risk and optional interview reason; formats recommendation.
Main function	<code>assess(case_id)</code>
Called by	Direct module/tests only
Calls	LLM: None. Database: Reads exit_cases/exit_interviews; inserts agent_runs.
Input	case_id
Processing	Reads already-computed case eligibility/risk and optional interview reason; formats recommendation.
LLM call	None.
Database operations	Reads exit_cases/exit_interviews; inserts agent_runs.
Output	Assessment dict.
Next step	No automatic next step.

Analytics agents

Dashboard Analytics (#14)

Not part of the live browser execution path

Field	Actual implementation
File	agents/analytics/analytics_agent.py
Purpose	Aggregates counts in Python, then narrates the already-computed stats.
Main function	run()
Called by	Direct module execution
Calls	LLM: ask_claude with dashboard_insights_system.md. Database: Reads exit_cases/exit_tasks; inserts analytics_insights(agent_type=dashboard_insights).
Input	No arguments; reads all cases/tasks
Processing	Aggregates counts in Python, then narrates the already-computed stats.
LLM call	ask_claude with dashboard_insights_system.md.
Database operations	Reads exit_cases/exit_tasks; inserts analytics_insights(agent_type=dashboard_insights).
Output	Graph state with stats and narrative.
Next step	HR layout can read the newest stored row.

SLA Escalation (#9) Not part of the live browser execution path

Field	Actual implementation
File	<code>agents/analytics/sla_escalation.py</code>
Purpose	Selects tasks at least five days overdue, resolves owner/impact, sends escalation notices.
Main function	<code>run(); find_breaches(...)</code>
Called by	Direct module; e2e_automation reuses finder/_escalate
Calls	LLM: None. Database: Reads exit_tasks/exit_cases/profiles; inserts agent_runs through escalation flow.
Input	All pending tasks/cases/profiles, or preloaded lists for pure finder
Processing	Selects tasks at least five days overdue, resolves owner/impact, sends escalation notices.
LLM call	None.
Database operations	Reads exit_tasks/exit_cases/profiles; inserts agent_runs through escalation flow.
Output	Breach list and escalation count.
Next step	No automatic scheduler; caller ends.

Exit Workflow Optimizer (#17) Not part of the live browser execution path

Field	Actual implementation
File	<code>agents/analytics/workflow_optimizer.py</code>
Purpose	Reuses SLA breach arithmetic and aggregates bottleneck counts.
Main function	<code>run()</code>
Called by	Direct module execution
Calls	LLM: ask_claude with workflow_optimizer_system.md. Database: Reads exit_tasks/exit_cases/profiles; inserts analytics_insights(workflow_optimizer).
Input	No arguments; reads pending workflow data
Processing	Reuses SLA breach arithmetic and aggregates bottleneck counts.
LLM call	ask_claude with workflow_optimizer_system.md.
Database operations	Reads exit_tasks/exit_cases/profiles; inserts analytics_insights(workflow_optimizer).
Output	Stats and narrative.
Next step	HR layout can read newest stored row.

Policy Compliance Auditor (#22)

Not part of the live browser execution path

Field	Actual implementation
File	<code>agents/analytics/policy_auditor.py</code>
Purpose	Checks SLA breach, missing approval, and skipped-step conditions, then formats a report.
Main function	<code>run()</code>
Called by	Direct module execution
Calls	LLM: ask_claude only when breaches exist. Database: Reads exit_cases/exit_tasks/agent_runs; inserts analytics_insights(policy_compliance_auditor).
Input	No arguments; reads active cases/tasks/runs
Processing	Checks SLA breach, missing approval, and skipped-step conditions, then formats a report.
LLM call	ask_claude only when breaches exist.
Database operations	Reads exit_cases/exit_tasks/agent_runs; inserts analytics_insights(policy_compliance_auditor).
Output	Report and narrative.
Next step	HR page reads stored narrative; live JS separately recomputes counts.

Predictive Attrition (#23)

Not part of the live browser execution path

Field	Actual implementation
File	<code>agents/analytics/attrition_agent.py</code>
Purpose	Combines average exit risk and trend-alert departments, counts current employees in signalled departments.
Main function	<code>run()</code>
Called by	Direct module execution
Calls	LLM: ask_claude only when signals exist. Database: Reads exit_cases/trend_alerts/profiles; inserts analytics_insights(predictive_attrition).
Input	No arguments; reads department-level signals
Processing	Combines average exit risk and trend-alert departments, counts current employees in signalled departments.
LLM call	ask_claude only when signals exist.
Database operations	Reads exit_cases/trend_alerts/profiles; inserts analytics_insights(predictive_attrition).
Output	Signals and narrative.
Next step	No automatic next step.

End-to-End Exit Automation (#24)

Not part of the live browser execution path

Field	Actual implementation
File	agents/hub/e2e_automation.py
Purpose	Graph composes service activation, supervisor, case-scoped SLA check, and persisted-gate finalization.
Main function	run(case_id, ...)
Called by	Direct module execution/tests
Calls	LLM: Only through delegated agents. Database: Reads/writes operational tables through reused functions; may update exit_cases.status and insert agent_runs.
Input	case_id, optional texts, simulate_rejection
Processing	Graph composes service activation, supervisor, case-scoped SLA check, and persisted-gate finalization.
LLM call	Only through delegated agents.
Database operations	Reads/writes operational tables through reused functions; may update exit_cases.status and insert agent_runs.
Output	Final E2EState with initiation, pipeline, SLA, outcome.
Next step	Caller receives completed or blocked outcome.

Relationship by execution path

```
LIVE BROWSER
submit-resignation Edge Function
  ↓ service.activate_case → Checklist/HR → Email Drafting
  ↓ human Manager gate
    └─ reject → escalation → HR action
    └─ approve → IT plan → human IT approval → IT execute/verify/audit
                                   ↓ Compliance (rechecked)
  ↓ human Finance RPC → Finance → Compliance → Risk
  ↓ human HR relieving-letter update → Email Drafting

CLI SUPERVISOR
Supervisor → Checklist/HR + KT Review + Smart Routing
  ↓ simulated Manager gate
  ↓ IT plan + Smart Routing → Compliance + Multi-System Clearance
  ↓ Finance → Exit Interview (optional) → Risk
```

SECTION 5

Supervisor / hub code

Field	Actual implementation
File	agents/hub/supervisor.py
Function	run_case(case_id, kt_text=None, interview_text=None, simulate_rejection=False)
Input	Case UUID plus optional KT/interview text and a simulated rejection flag.
State	SupervisorState : case_id, kt_text, interview_text, simulate_rejection, log.
Routing logic	Entry is hr. A conditional edge after manager_gate maps approved→it and rejected→escalate.
Condition	Only simulate_rejection; no browser/database approval check.
Next nodes	Normal: it→compliance→finance→assess. Rejection: escalate→END.
Database interaction	Each stage delegates to agents; _record appends in-memory log and inserts agent_runs. _activate conditionally updates exit_cases.
Return value	The final state returned by supervisor_graph.invoke .

```
case_id + optional texts + simulate_rejection
  ↓ _hr_stage: activate; checklist; optional KT review; HR routing
  ↓ _manager_gate: manager routing; record approved/rejected
  ↓ _route_after_manager
    └─ "rejected" → _escalate → insert open escalation task → END
    └─ "approved" → _it_stage → plan + IT routing
      ↓ _compliance_stage → compliance + multi-system status
      ↓ _finance_stage → finance task/status
      ↓ _assess_stage → optional interview + risk
      ↓ final SupervisorState
```

agents/hub/supervisor.py

Lines 45–68 · State and durable recording

```
45 # TypedDict is a Python way to define a dictionary with named, typed fields --
46 # think of it as a structured record that gets passed through every graph node.
47 class SupervisorState(TypedDict):
48     case_id: str          # the Supabase row id for this exit case
49     kt_text: str | None   # optional Knowledge Transfer document text
50     interview_text: str | None # optional exit interview transcript
51     simulate_rejection: bool # if True, the manager gate rejects the case
52     log: list[str]        # accumulates log messages as the pipeline runs
53
54
```

```

55 def _record(state: SupervisorState, stage: str, detail: str) -> None:
56     """Append to state["log"] (already existed) AND persist a row so the HR
57     dashboard's agent-activity view (B2) has something to read -- the
58     terminal trace alone (B1) is stdout-only."""
59     state["log"].append(detail)
60     db.table("agent_runs").insert({"case_id": state["case_id"], "stage": stage, "detail": detail}).execute()
61     log_db("insert", "agent_runs", rows=1, detail=detail)
62
63
64 def _activate(case_id: str) -> None:
65     """Move a case off the default 'open' the moment orchestration starts, the
66     same transition service.activate_case makes on the browser path. Without
67     this a CLI-driven case stays 'open' on the HR dashboard even after a full
68     successful run.

```

Lines 45–50: The shared state contains only routing inputs and an accumulating log; agents reload case data from Supabase.

Lines 53–58: `_record` mutates `state.log` and inserts the same event into `agent_runs`.

Lines 61–68: `_activate` changes only an `open` case, preventing a rerun from downgrading later status.

agents/hub/supervisor.py

Lines 87–157 · Nodes and branch behavior

```

87 def _hr_stage(state: SupervisorState) -> SupervisorState:
88     _activate(state["case_id"])
89     checklist_generator_agent.generate(state["case_id"])
90     if state.get("kt_text"):
91         kt_document_reviewer_agent.review(state["case_id"], state["kt_text"])
92     routing = smart_routing.pick_approver(state["case_id"], "hr")
93     _record(state, "hr", "hr: checklist + kt-review done")
94     _record(state, "hr", f"smart routing: {routing['reason']}")
95     return state
96
97
98 @traced_node("Supervisor -- manager gate")
99 def _manager_gate(state: SupervisorState) -> SupervisorState:
100     routing = smart_routing.pick_approver(state["case_id"], "manager")
101     _record(state, "manager", f"smart routing: {routing['reason']}")
102     _record(state, "manager", "rejected" if state["simulate_rejection"] else "approved")
103     return state
104
105
106 # Routing function: returns a string that selects which branch to follow next.
107 # LangGraph calls this after the manager_gate node; the return value ("approved"
108 # or "rejected") maps to the corresponding edge in add_conditional_edges below.

```

```

109 def _route_after_manager(state: SupervisorState) -> str:
110     return "rejected" if state["simulate_rejection"] else "approved"
111
112
113 @traced_node("Supervisor -- escalate")
114 def _escalate(state: SupervisorState) -> SupervisorState:
115     db.table("exit_tasks").insert({
116         "case_id": state["case_id"], "stage": "manager", "status": "pending",
117         "title": "Escalated: manager rejected KT plan -- HR review needed",
118         "escalation_state": "open",
119     }).execute()
120     log_db("insert", "exit_tasks", rows=1, detail="escalation")
121     _record(state, "escalate", "escalated to HR, stopping short of IT/finance")
122     return state
123
124
125 @traced_node("Supervisor -- IT stage")
126 def _it_stage(state: SupervisorState) -> SupervisorState:
127     it_deprovisioning_agent.generate(state["case_id"])
128     routing = smart_routing.pick_approver(state["case_id"], "it")
129     _record(state, "it", "it: deprovisioning plan done")
130     _record(state, "it", f"smart routing: {routing['reason']}")
131     return state
132
133
134 @traced_node("Supervisor -- compliance stage")
135 def _compliance_stage(state: SupervisorState) -> SupervisorState:
136     result = compliance_agent.run_for_case(state["case_id"])["result"]
137     detail = "compliance: cleared" if result["cleared"] else f"compliance: blocked -- {result['blocking_reasons']}"
138     _record(state, "compliance", detail)
139     status = multi_system_clearance consolidate(state["case_id"])
140     _record(state, "compliance", f"multi-system clearance: {status['overall']} "
141                                f"(it={status['it_asset_management']['status']}, hrms={status['hrms']['status']}, "
142                                f"finance={status['finance']['status']})")
143     return state
144
145
146 @traced_node("Supervisor -- finance stage")
147 def _finance_stage(state: SupervisorState) -> SupervisorState:
148     finance_agent.check_clearance(state["case_id"])
149     _record(state, "finance", "finance: clearance checked")
150     return state

```

```

151
152
153 @traced_node("Supervisor -- assess")
154 def _assess_stage(state: SupervisorState) -> SupervisorState:
155     if state.get("interview_text"):
156         run_per_case(state["case_id"], state["interview_text"])
157     risk_agent.run_for_case(state["case_id"])

```

Lines 87–96: HR activation/checklist are unconditional; KT review is conditional on text; routing is recorded, not assigned to the case row.

Lines 98–112: The manager node records a simulated decision and the routing function converts the boolean into an edge key.

Lines 115–124: Rejection inserts one open escalation task and terminates this graph branch.

Lines 126–157: The approved branch generates IT work, evaluates compliance/system state, evaluates finance, then optionally analyzes the interview and always scores risk.

agents/hub/supervisor.py

Lines 165–197 · Graph construction and invocation

```

165 # add_edge("a", "b")          -- after "a" runs, always run "b".
166 # add_conditional_edges       -- after "manager_gate", call _route_after_manager()
167 #                             and jump to whichever node its return value names.
168 # compile() locks the graph so it can be invoked (run) later.
169 _graph = StateGraph(SupervisorState)
170 _graph.add_node("hr", _hr_stage)
171 _graph.add_node("manager_gate", _manager_gate)
172 _graph.add_node("escalate", _escalate)
173 _graph.add_node("it", _it_stage)
174 _graph.add_node("compliance", _compliance_stage)
175 _graph.add_node("finance", _finance_stage)
176 _graph.add_node("assess", _assess_stage)
177
178 _graph.set_entry_point("hr")          # the pipeline always starts at the HR stage
179 _graph.add_edge("hr", "manager_gate")
180 _graph.add_conditional_edges("manager_gate", _route_after_manager, {"approved": "it", "rejected": "escalate"})
181 _graph.add_edge("it", "compliance")
182 _graph.add_edge("compliance", "finance")
183 _graph.add_edge("finance", "assess")
184 _graph.set_finish_point("assess")     # the pipeline ends after risk assessment
185 _graph.add_edge("escalate", END)      # a rejected case stops here (no IT/finance)
186 supervisor_graph = _graph.compile()  # compile() makes the graph executable
187
188
189 def run_case(case_id: str, *, kt_text: str | None = None, interview_text: str | None = None,
190             simulate_rejection: bool = False) -> dict:
191     return supervisor_graph.invoke({

```

```
192         "case_id": case_id,  
193         "kt_text": kt_text,  
194         "interview_text": interview_text,  
195         "simulate_rejection": simulate_rejection,  
196         "log": [],  
197     })
```

Lines 165–175: Each Python function becomes a named LangGraph node.

Lines 177–185: Edges define the exact order; only `manager_gate` is conditional.

Lines 186–197: Compilation creates the runnable graph; `run_case` supplies the initial state and returns the final state.

SECTION 6

Service / API code

Supabase Edge Function endpoints

HTTP method	Endpoint / handler	Input	Validation	Calls	Database changes	Response
POST	<code>submit-resignation</code> <code>Deno.serve</code>	<code>last_working_day</code> , optional reason; Authorization JWT	POST only; date string required; JWT user must exist; profile.role must be employee; returns existing case idempotently.	Supabase Auth, profiles lookup.	Inserts <code>exit_cases</code> only when absent.	{ ok:true, case } or structured error.
POST	<code>ask</code> <code>Deno.serve</code>	question string	POST only; nonempty string. Function code does not call <code>getUser</code> ; platform JWT enforcement is outside this file.	Embedding endpoint, <code>match_exit_docs</code> RPC, message endpoint.	No database writes.	{ answer, sources, refused, general }.
POST	<code>forward-to-hr</code> <code>Deno.serve</code>	question string; Authorization JWT	POST only; question required; JWT validated with <code>getUser</code> .	profiles, Gmail SMTP; fallback case lookup.	On SMTP failure inserts <code>agent_runs</code> fallback row.	{ ok:true } or { ok:true, delayed:true, message }.

Local Python endpoints

Method	Endpoint	Function	Required input	Validation / gate	Agent or work called	Database effect	Response
POST	<code>/activate-exit</code>	<code>activate_case</code>	<code>case_id</code>	Case must exist.	<code>hr_agent.generate_checklist</code> ; <code>resignation_notice</code>	<code>tasks</code> ; <code>exit_cases.status</code> ; email audit	ok, checklist, manager_notice
POST	<code>/submit-exit-interview</code>	<code>submit_exit_interview</code>	<code>case_id</code>	Interview row must exist.	<code>exit_intel_agent.run_per_case</code>	updates <code>exit_interviews</code>	ok, analysis
POST	<code>/issue-relieving-letter</code>	<code>issue_relieving_letter</code>	<code>case_id</code>	Case must exist; final authorization occurred in prior browser DB update.	<code>relieving_letter_notice</code>	email audit only	ok, notice
POST	<code>/validate-document</code>	<code>validate_document</code>	<code>case_id</code> , <code>document_id</code>	<code>document_id</code> required; row matched to case.	<code>doc_collection.validate_one</code> ; <code>compliance_agent.run_for_case</code>	<code>case_documents</code> , compliance task/checks, audit	ok, validation, compliance

Method	Endpoint	Function	Required input	Validation / gate	Agent or work called	Database effect	Response
POST	/execute-deprovisioning	execute_deprovisioning	case_id	Only stage=it status=done tasks are selected by agent.	it_deprovisioning_agent.execute_approved_tasks; compliance	agent_runs, compliance rows	ok, executed, compliance
POST	/finance-settle-check	finance_settle_check	case_id	Finance RPC already set flag; agents recompute from DB.	finance; compliance; risk	finance/compliance tasks; checks; case risk; audits	ok + three results
POST	/manager-approve	manager_approve	case_id	Case exists; no open escalation; KT tasks exist; all are done.	IT plan; compliance	manager/IT audit, IT tasks, compliance rows	advanced true/false + reason/results
POST	/reject-manager-task	reject_manager_task	case_id, task_id, reason	Task belongs to case, manager stage, not escalation; reason nonempty.	Direct mirror of escalation write.	exit_tasks escalation; agent_runs	ok or validation error
POST	/escalation-audit	log_escalation_transition	case_id, task_id, action, actor_name	action is rerouted/resolved; browser RLS already performed state change.	Audit; closes task only for resolved state.	agent_runs; possibly exit_tasks.status	ok

Authentication boundary: `agents/service.py` binds to localhost and performs no JWT authentication. It uses the service-role client. Its checks are workflow/data checks, not caller-identity checks.

One complete request: manager approval

```

ManagerPages.useApprove.approveTask(task)
  ↓ anon-key UPDATE exit_tasks SET status='done' WHERE id=task.id
  ↓ Postgres RLS 0008 validates manager ownership + forward-only status
  ↓ POST /manager-approve { case_id }
  ↓ Handler.do_POST parses JSON and dispatches manager_approve
  ↓ read manager tasks + exclude Escalated* rows
  ↓ open escalation? stop
  ↓ no KT tasks or any pending? return advanced=false
  ↓ insert manager approved agent_run if latest decision is not approved
  ↓ it_deprovisioning_agent.generate → it_agent.generate_plan
  ↓ compliance_agent.run_for_case
  ↓ return { ok:true, advanced:true, it_plan, compliance }
  ↓ ManagerPages reloads data

```

```

323     if action == "resolved":
324         closed = db.table("exit_tasks").update({"status": "done"}) \
325             .eq("id", task_id).eq("escalation_state", "resolved").execute().data
326         if closed:
327             log_db("update", "exit_tasks", rows=1, detail="escalation resolved -> status done")
328     return {"ok": True}
329
330
331 # — HTTP request handler —————
332 # BaseHTTPRequestHandler is Python's built-in way to handle HTTP requests.
333 # do_OPTIONS / do_POST are method names the framework calls automatically when
334 # a request with that HTTP method arrives. CORS headers are needed because the
335 # browser blocks cross-origin requests (from localhost:5173 to localhost:8787)
336 # unless the server explicitly allows them.
337 class Handler(BaseHTTPRequestHandler):
338     def _cors(self) -> None:
339         self.send_header("Access-Control-Allow-Origin", ALLOWED_ORIGIN)
340         self.send_header("Access-Control-Allow-Methods", "POST, OPTIONS")
341         self.send_header("Access-Control-Allow-Headers", "Content-Type")
342
343     def _json(self, status: int, payload: dict) -> None:
344         body = json.dumps(payload).encode("utf-8")
345         self.send_response(status)
346         self._cors()
347         self.send_header("Content-Type", "application/json")
348         self.end_headers()
349         self.wfile.write(body)
350
351     def do_OPTIONS(self) -> None: # noqa: N802 (BaseHTTPRequestHandler naming)
352         self.send_response(204)
353         self._cors()
354         self.end_headers()
355
356     def do_POST(self) -> None: # noqa: N802
357         # Route table: maps each URL path to the function that handles it.
358         # lambda body: ... delays execution so the function only runs after
359         # the request body has been parsed and we know it's valid.
360         routes = {
361             "/activate-exit": lambda body: activate_case(body["case_id"]),
362             "/submit-exit-interview": lambda body: submit_exit_interview(body["case_id"]),
363             "/issue-relieving-letter": lambda body: issue_relieving_letter(body["case_id"]),

```

```

364         "/validate-document": lambda body: validate_document(body["case_id"], body.get("document_id")),
365         "/execute-deprovisioning": lambda body: execute_deprovisioning(body["case_id"]),
366         "/finance-settle-check": lambda body: finance_settle_check(body["case_id"]),
367         "/manager-approve": lambda body: manager_approve(body["case_id"]),
368         "/reject-manager-task": lambda body: reject_manager_task(body["case_id"], body.get("task_id"),
body.get("reason")),
369         "/escalation-audit": lambda body: log_escalation_transition(
370             body["case_id"], body.get("task_id"), body.get("action"), body.get("actor_name")
371         ),
372     }
373     if self.path not in routes:
374         self._json(404, {"error": "not found"})
375         return
376     length = int(self.headers.get("Content-Length", 0))
377     try:
378         body = json.loads(self.rfile.read(length) or b"{}")
379     except json.JSONDecodeError:
380         self._json(400, {"error": "invalid JSON body"})
381         return
382     if not body.get("case_id"):
383         self._json(400, {"error": "case_id required"})
384         return
385     handler = routes[self.path]
386     try:
387         result = handler(body)
388     except Exception as exc: # pipeline/LLM/DB failure -- report, don't crash the service
389         self._json(500, {"error": str(exc)})
390         return
391     self._json(200, result)
392
393     def log_message(self, format: str, *args) -> None: # noqa: A002
394         print("[agent-service]", format % args)
395
396
397     def main() -> None:

```

Lines 323–337: Helper methods add CORS and encode JSON; OPTIONS answers the browser preflight.

Lines 340–355: A literal route table maps all nine paths to Python functions.

Lines 356–373: Unknown paths, invalid JSON, and missing case_id return 4xx responses.

Lines 374–381: Runtime exceptions become JSON 500 responses without stopping the server.

Lines 387–397: `ThreadingHTTPServer` listens only on localhost:8787 and serves until stopped.

SECTION 7

RAG code path

```
scripts/exit_policy.md
  ↓ scripts/ingest_docs.js · chunkText
  ↓ one chunk per $n.n section
  ↓ embed(chunk) → Portkey /v1/embeddings
  ↓ exit_docs(source, section, content, embedding vector(1536))
  ↓ employee question → supabase/functions/ask/index.ts
  ↓ embed(question)
  ↓ db.rpc('match_exit_docs', query_embedding, match_count=4)
  ↓ 0003_rag.sql orders by embedding <=> query_embedding
  ↓ context = [section] + content for each returned chunk
  ↓ askModel(context, question)
  ↓ parseReply → answer + named sections + scope
  ↓ citations matched to retrieved chunks; threshold 0.35
  ↓ { answer, sources, refused, general } → EmployeePages
```

Question	Actual answer
Where are documents stored?	The source policy is <code>scripts/exit_policy.md</code> . Each chunk and vector is stored in PostgreSQL table <code>exit_docs</code> . Employee uploads are separate: Storage bucket <code>exit-documents</code> plus metadata in <code>case_documents</code> ; those uploads are not RAG sources.
Where are embeddings generated?	Both ingestion and query embedding call the configured Portkey <code>/v1/embeddings</code> URL. The model route is <code>PORTKEY_VIRTUAL_KEY</code> .
Where are vectors stored?	<code>exit_docs.embedding vector(1536)</code> , indexed with HNSW cosine operators.
How does similarity search happen?	<code>match_exit_docs</code> returns <code>1 - (embedding <=> query_embedding)</code> , ordered by distance, limited to four rows.
How is context passed?	The Edge Function joins retrieved rows as <code>[section] content</code> and sends that plus the question in one user message.
How are citations produced?	The model returns a <code>sections</code> array. Code matches those names case-insensitively to retrieved rows, deduplicates them, and falls back to only the best chunk. No citation is returned for general/refused output or when best similarity is below 0.35.
How does access/privacy filtering work?	RLS is enabled on <code>exit_docs</code> with no browser policies. The Edge Function uses the service-role key to retrieve. The retrieval query has no employee, department, document, or ACL filter: every row in <code>exit_docs</code> is in the same policy corpus.

No RAG over uploaded case documents. `case_documents` files are OCR-validated for compliance. They are never chunked into `exit_docs` and never supplied to `askModel`.

scripts/ingest_docs.jsLines 22–47 · Chunk and label policy sections

22 const SOURCE = 'exit_policy.md'

23

```

24 // One chunk per "$n.n Title" section. sectionOf() labels a chunk with only its
25 // FIRST section, so packing two sections together cites the first one as the
26 // source of the second one's text — the sections are self-contained, so a
27 // clean 1:1 split is what keeps citations honest. A section that runs to
28 // several paragraphs stays whole.
29 function chunkText(text) {
30   // Split the markdown on blank lines to get individual paragraphs.
31   const paragraphs = text.split(/\n\s*\n/).map((p) => p.trim()).filter(Boolean)
32   const chunks = []
33   for (const p of paragraphs) {
34     // A paragraph starting with $1.2 Title begins a new policy section — start a new chunk.
35     // Any paragraph without that marker is a continuation of the previous section, so append it.
36     if (chunks.length === 0 || /^$[\d.]+\s+/.test(p)) chunks.push(p)
37     else chunks[chunks.length - 1] += `\n\n${p}`
38   }
39   return chunks
40 }
41
42 function sectionOf(chunk) {
43   // Extract just the section title (the part after "$1.2 ") from the first line of a chunk.
44   // Returns null for chunks that don't start a section (shouldn't happen after chunkText).
45   const match = chunk.match(/^$[\d.]+\s+(.+)$/m)
46   return match ? match[1].trim() : null
47 }

```

Lines 22–29: The fixed source name and comment establish one row group per local markdown file.

Lines 29–40: Blank lines form paragraphs; a paragraph beginning with the section marker starts a new chunk; continuations append.

Lines 42–47: `sectionOf` extracts the citation label stored beside the text.

scripts/ingest_docs.js

Lines 49–94 · Embed and replace stored vectors

```

49 async function embed(input) {
50   // POST to Portkey's embeddings endpoint — same API shape as OpenAI's /v1/embeddings.
51   // embedModel is the PORTKEY_VIRTUAL_KEY which maps to text-embedding-3-small (1536-dim).
52   const res = await fetch(`${portkeyUrl}/v1/embeddings`, {
53     method: 'POST',
54     headers: {
55       Authorization: `Bearer ${portkeyKey}`,
56       'Content-Type': 'application/json',
57     },
58     body: JSON.stringify({ model: embedModel, input }),
59   })
60   if (!res.ok) throw new Error(`embeddings ${res.status}: ${await res.text()}`)

```

```

61  const json = await res.json()

62  // The API returns an array of embedding objects; [0].embedding is the float vector.

63  return json.data[0].embedding

64 }

65

66 async function main() {

67   const text = readFileSync(new URL('./exit_policy.md', import.meta.url), 'utf8')

68   const chunks = chunkText(text)

69   console.log(`chunked ${SOURCE} into ${chunks.length} chunks`)

70

71   // Delete all previous rows for this source first, so re-running stays idempotent

72   // (no duplicate chunks if the policy doc is updated and re-ingested).

73   const { error: delErr } = await db.from('exit_docs').delete().eq('source', SOURCE)

74   if (delErr) throw new Error(`exit_docs.delete: ${delErr.message}`)

75

76   for (const chunk of chunks) {

77     const embedding = await embed(chunk)

78     // Fail loudly if the embedding dimension doesn't match the pgvector column (1536).

79     // A silent dimension mismatch would silently write unusable vectors.

80     if (embedding.length !== 1536) {

81       throw new Error(`embedding dim mismatch: got ${embedding.length}, expected 1536`)

82     }

83     // Insert the chunk text + its vector embedding together – the /ask Edge Function

84     // queries this table using match_exit_docs() (pgvector cosine similarity) to find

85     // the most relevant policy sections for a given employee question.

86     const { error } = await db

87       .from('exit_docs')

88       .insert({ source: SOURCE, section: sectionOf(chunk), content: chunk, embedding })

89     if (error) throw new Error(`exit_docs.insert: ${error.message}`)

90   }

91   console.log(`ingested ${chunks.length} chunks into exit_docs`)

92 }

93

94 main()

```

Lines 49–64: `embed` posts the raw chunk and returns the first embedding vector.

Lines 66–74: `main` reads the local policy and deletes prior rows for the same source.

Lines 76–90: Every chunk is embedded, dimension-checked, and inserted with source/section/content.

Lines 94 onward: Process exit status represents success or failure for this manual script.

supabase/migrations/0003_rag.sql

Lines 22–44 · Similarity RPC

```

22 create or replace function match_exit_docs(

23   query_embedding vector(1536),

```

```

24     match_count int default 4
25 )
26 returns table (
27     id          bigint,
28     source      text,
29     section     text,
30     content     text,
31     similarity double precision
32 )
33 language sql stable
34 as $$
35     select
36         id,
37         source,
38         section,
39         content,
40         1 - (embedding <=> query_embedding) as similarity
41     from exit_docs
42     order by embedding <=> query_embedding
43     limit match_count;
44 $$;

```

Lines 22–28: Input is a 1536-value vector and optional top-k count; output columns include similarity.

Lines 32–43: The SELECT converts cosine distance to similarity, sorts nearest first, and limits the result.

Security detail: The function is intentionally not SECURITY DEFINER. The ask function can call it because its service-role client bypasses RLS.

supabase/functions/ask/index.ts

Lines 180–255 · Retrieve, ground, classify, and cite

```

180 Deno.serve(async (req) => {
181     if (req.method === 'OPTIONS') {
182         return new Response(null, { headers: CORS_HEADERS })
183     }
184     if (req.method !== 'POST') {
185         return new Response('Method not allowed', { status: 405, headers: CORS_HEADERS })
186     }
187
188     try {
189         const { question } = await req.json()
190         if (!question || typeof question !== 'string') {
191             return new Response(JSON.stringify({ error: 'question is required' }), {
192                 status: 400,
193                 headers: { ...CORS_HEADERS, 'Content-Type': 'application/json' },
194             })

```

```

195     }
196
197     // Step 1: Turn the user's question into a vector (list of numbers).
198     const queryEmbedding = await embed(question)
199
200     // Step 2: Find the 4 policy chunks whose vectors are closest to the question.
201     // match_exit_docs is a PostgreSQL function (0003_rag.sql) that does this
202     // vector similarity search using the pgvector extension.
203     const { data: chunks, error } = await db.rpc('match_exit_docs', {
204         query_embedding: queryEmbedding,
205         match_count: 4,
206     })
207     if (error) throw new Error(`match_exit_docs: ${error.message}`)
208
209     if (!chunks || chunks.length === 0) {
210         // No relevant policy found -- return a safe refusal rather than letting the
211         // AI hallucinate an answer with no grounding.
212         return new Response(JSON.stringify(REFUSAL), {
213             headers: { ...CORS_HEADERS, 'Content-Type': 'application/json' },
214         })
215     }
216
217     // Step 3: Build a "context" string by joining the retrieved chunks.
218     // Each chunk is labelled with its section title so the AI can cite it.
219     const context = chunks
220         .map((c: { section: string | null; source: string; content: string }) => `${c.section || c.source}\n${c.content}`)
221         .join('\n\n')
222
223     // Step 4: Ask the AI to answer the question using only the retrieved context.
224     const { answer, sections, scope } = parseReply(await askModel(context, question))
225     // No scope means the model ignored the JSON contract -- fall back to the
226     // wording heuristic so a refusal still reaches the Forward-to-HR path.
227     const effectiveScope: Scope = scope ?? (looksLikeRefusal(answer) ? 'refused' : 'policy')
228     const refused = effectiveScope === 'refused'
229     // General-knowledge answer: not grounded in the policy, so it carries no
230     // citations and the UI labels it as guidance rather than company rule.
231     const general = effectiveScope === 'general'
232
233     // Cite only what the answer actually used. If the model named nothing
234     // usable, fall back to the single best-matching chunk rather than listing
235     // every chunk retrieval happened to return.
236     let sources: { source: string; section: string | null }[] = []

```

```

237   if (effectiveScope === 'policy' && chunks[0].similarity >= SIMILARITY_THRESHOLD) {
238       const used = new Set(sections.map((s) => s.trim().toLowerCase()).filter(Boolean))
239       const matched = chunks.filter((c: { source: string; section: string | null }) =>
240           used.has(String(c.section ?? c.source).trim().toLowerCase()),
241       )
242       const seen = new Set<string>()
243       sources = (matched.length > 0 ? matched : [chunks[0]])
244           .filter((c: { source: string; section: string | null }) => {
245               const key = `${c.source}|${c.section ?? ''}`
246               if (seen.has(key)) return false
247               seen.add(key)
248               return true
249           })
250       .map((c: { source: string; section: string | null }) => ({ source: c.source, section: c.section }))
251   }
252
253   return new Response(JSON.stringify({ answer, sources, refused, general }), {
254       headers: { ...CORS_HEADERS, 'Content-Type': 'application/json' },
255   })

```

Lines 180–195: The handler accepts OPTIONS/POST and validates question text.

Lines 197–215: It embeds once, retrieves four chunks, and refuses immediately if retrieval is empty.

Lines 217–227: Chunks become labelled context; the LLM reply is parsed into answer/sections/scope.

Lines 228–251: Scope and similarity decide whether citations are allowed; section names select only actually used chunks.

Lines 253–255: The structured result is returned to React.

SECTION 8

Database flow

Table / storage	Purpose	Read by	Written by	Important fields	Relationships
profiles	Auth-user role and identity.	App/layouts, submit/forward Edge Functions, routing, risk, notifications.	Seeds/admin; OOO setting.	id, role, employee_id, department, out_of_office	id→auth.users; employee_id→exit_cases.employee_id.
exit_cases	One offboarding case and final HR-only assessment.	Service, all operational agents, role views, HR.	submit-resignation; service activation; risk; finance RPC; HR relieving update.	employee_id, manager_id, hr_id, last_working_day, status, finance_cleared, risk_*, relieving_*	Parent of tasks/interviews/docs/runs/checks.
exit_tasks	All workflow items and gates.	Every role layout; service; finance/compliance/IT agents.	HR/IT/finance/compliance agents; employee/manager/IT RLS updates; escalation code.	Base migration: case_id, stage, title, status, due_date, kt_event_id. Code also expects reason and escalation_state.	Many tasks belong to one exit_case.
exit_interviews	Raw employee answers plus LLM analysis.	Exit intelligence, risk, HR.	Employee submit; exit_intel update/insert.	case_id, raw answers, summary, sentiment, themes, rehire_*	One logical interview per case.
case_documents	Metadata and OCR validation for uploads.	Employee/HR UI, doc collection, compliance, multi-system clearance.	Employee insert; doc collection update.	case_id, doc_type, file_path, status, validation_detail	File bytes live in exit-documents Storage.
kt_reviews	KT summary/gaps, restricted to HR/manager.	HR/manager if queried.	hr_agent KT persist.	case_id, summary, gaps, complete	Belongs to exit_case.
compliance_checks	One row per explicit compliance item expected by Python and verification code.	Regression verification script; no current React query reads it.	compliance_agent upsert.	Code expects case_id, item, status, source, evidence, failure_reason, checked_at.	Code assumes a unique case_id,item key, but no CREATE TABLE appears in migrations 0001–0030.
agent_runs	Durable activity/audit events.	HR layout; compliance manager signal; IT view verification subquery.	Supervisor, service, risk, email, routing, document, IT, analytics.	0009 defines case_id, stage, detail. Code also writes/reads agent, status, metadata.	Belongs to exit_case; the extra audit columns are not added by migrations 0001–0030.
trend_alerts	Repeated exit-interview themes.	HR, attrition agent.	exit_intel longitudinal.	theme, department, severity, detail	Derived across cases.
analytics_insights	Stored narratives and structured stats.	HR layout.	Four analytics agents.	narrative, stats, agent_type, created_at	Rows distinguished by agent_type.
exit_docs	RAG chunk store.	match_exit_docs via ask service-role client.	ingest_docs.js.	source, section, content, embedding	Independent policy corpus; no case_id.

Table / storage	Purpose	Read by	Written by	Important fields	Relationships
Storage: exit-documents	Employee-uploaded file bytes.	doc_collection download.	Employee Documents upload.	path = case_id/docType-timestamp.ext	case_documents.file_path points here.

Migration drift visible in this repository: Python/React code uses `compliance_checks`; `exit_tasks.reason/escalation_state`; and `agent_runs.agent/status/metadata`. No migration in `supabase/migrations/0001-0030` creates those objects/columns. Migration 0026 even updates `escalation_state` and 0025 reads `agent_runs.status/metadata`, so a fresh database built only from these files is not self-contained at those points.

Operational write chain

```
submit-resignation → exit_cases
service.activate_case → exit_cases.status + exit_tasks(hr/manager) + agent_runs(email)
manager approval → exit_tasks(manager done) → agent_runs(manager) → exit_tasks(it)
IT approval → exit_tasks(it done) → agent_runs(it_deprovisioning_execution)
compliance_agent → exit_tasks(compliance) + compliance_checks
finance RPC → exit_cases.finance_cleared/dues_note
finance_agent → exit_tasks(finance)
risk_agent → exit_cases.risk_score/risk_level/rehire_eligible + agent_runs
HR issue → exit_cases.relieving_* + status=completed → agent_runs(email)
```

Views and RPCs used by browser roles

Object	Consumer	What it exposes/changes
employee_exit_view	EmployeeLayout	Own safe case fields, including relieving state; excludes risk fields.
manager_case_view	ManagerLayout	Cases assigned to <code>auth.uid()</code> with no HR-only assessment fields.
it_task_view	ItLayout	IT tasks plus case display fields and latest execution verification status.
finance_case_view	FinanceLayout	Safe case/finance fields; excludes risk and interview fields.
employee_interview_status_view	EmployeePages.ExitInterview	Only whether the current employee has submitted.
finance_mark_dues_settled	FinancePages.useSettle	Security-definer role check; sets <code>finance_cleared=true</code> and dues note.
finance_reject_dues	FinancePages.useReject	Security-definer role check; clears finance flag and records rejection reason.
exit_case_cleared_for_relieving	RLS WITH CHECK for HR update	Requires <code>finance_cleared</code> , all four visible stages done, and all four stages present.
match_exit_docs	ask Edge Function	Returns nearest policy chunks to a query vector.

SECTION 9

Line-by-line source walkthrough

Each excerpt below is copied directly from the current source file. The line notes explain the control flow and data effects of that exact block.

9.1 React startup and routing

src/main.jsx

Lines 1–13 · Mount the application

```

1 import { StrictMode } from 'react'
2 import { createRoot } from 'react-dom/client'
3 import '@tabler/icons-webfont/dist/tabler-icons.min.css'
4 import './styles/tokens.css'
5 import './styles/dashboards.css'
6 import App from './App.jsx'
7
8 createRoot(document.getElementById('root')).render(
9   <StrictMode>
10     <App />
11   </StrictMode>
12 )
13

```

Lines 1–7: Import React root APIs, `App`, icon CSS, and the two global stylesheets.

Lines 9–14: Find `#root` in `index.html` and render `App` inside `StrictMode`.

src/App.jsx

Lines 19–64 · Resolve session and role

```

19 // useState stores values that can change over time. When they change, React
20 // re-renders the component automatically.
21 // undefined = "still checking auth"; null = "checked, not logged in".
22 const [session, setSession] = useState(undefined) // undefined = loading
23 const [role, setRole] = useState(null) // "employee" | "hr" | "manager" | "it" | null
24 const [devRole, setDevRole] = useState(null) // dev-only override role
25
26 // useEffect runs code after the component renders. The empty [] means it runs
27 // only once (on first render), not on every update.
28 useEffect(() => {
29   // Get the current session (e.g. from a saved cookie/token after a page refresh).
30   supabase.auth.getSession().then(({ data }) => setSession(data.session))
31   // Subscribe to future auth changes (login, logout) and keep session in sync.
32   const { data: sub } = supabase.auth.onAuthStateChange((_event, s) => setSession(s))
33   // Cleanup: unsubscribe when the component is removed from the page.
34   return () => sub.subscription.unsubscribe()

```

```

35   }, [])
36
37   // Whenever session changes, look up the user's role in the profiles table.
38   // [session] means: re-run this effect every time the `session` value changes.
39   useEffect(() => {
40     if (!session) {
41       setRole(null)
42       return
43     }
44     supabase
45       .from('profiles')
46       .select('role')
47       .eq('id', session.user.id) // match the logged-in user's auth ID
48       .single()
49       .then(({ data }) => setRole(data?.role ?? null))
50   }, [session])
51
52   // Show nothing while auth is loading (avoids a flash of the wrong screen).
53   if (session === undefined || (session && role === null)) return null
54   if (!session) return <LoginPage />
55
56   const activeRole = ROLE_SWITCHER_ENABLED && devRole ? devRole : role
57
58   return (
59     <>
60       {ROLE_SWITCHER_ENABLED && <RoleSwitcher role={devRole ?? role} onChange={setDevRole} />}
61       <div className="frame">
62         <BrowserRouter>
63           <AppRoutes role={activeRole} session={session} />
64         </BrowserRouter>

```

Lines 19–23: Three states distinguish authentication loading, resolved role, and optional dev override.

Lines 27–34: Initial session lookup and auth-change subscription keep session state current; cleanup unsubscribes.

Lines 38–50: When logged in, read only `profiles.role` for the authenticated user.

Lines 53–64: Render nothing while resolving, `LoginPage` without a session, otherwise role-aware routes.

src/routes/AppRoutes.jsx

Lines 20–109 · Role-guarded routes

```

20
21 import FinanceLayout from '../finance/FinanceLayout'
22 import * as Finance from '../finance/FinancePages'
23
24 // role is derived from the logged-in user's profiles.role column in Supabase.

```

```

25 // session is the Supabase Auth session object (holds the user's JWT).
26 export default function AppRoutes({ role, session }) {
27   return (
28     <Routes>
29       {/* /employee/resignation is outside the main employee layout because
30        employees who haven't resigned yet land here directly – no sidebar needed. */}
31       <Route
32         path="/employee/resignation"
33         element={role === 'employee' ? <Employee.Resignation session={session} /> : <Navigate to={`/${role}`} replace />}
34       />
35
36       <Route
37         path="/employee"
38         element={role === 'employee' ? <EmployeeLayout session={session} /> : <Navigate to={`/${role}`} replace />}
39       >
40         <Route index element={<Employee.Dashboard />} />
41         <Route path="my-exit" element={<Employee.MyExit />} />
42         <Route path="tasks" element={<Employee.Tasks />} />
43         <Route path="documents" element={<Employee.Documents />} />
44         <Route path="knowledge-transfer" element={<Employee.KnowledgeTransfer />} />
45         <Route path="exit-interview" element={<Employee.ExitInterview />} />
46         <Route path="timeline" element={<Employee.Timeline />} />
47         <Route path="help" element={<HelpPage role="Employee" />} />
48         <Route path="*" element={<Navigate to="" replace />} />
49       </Route>
50
51       <Route
52         path="/manager"
53         element={role === 'manager' ? <ManagerLayout session={session} /> : <Navigate to={`/${role}`} replace />}
54       >
55         <Route index element={<Manager.Dashboard />} />
56         <Route path="my-team" element={<Manager.MyTeam />} />
57         <Route path="exiting-reports" element={<Manager.ExitingReports />} />
58         <Route path="kt-approvals" element={<Manager.KtApprovals />} />
59         <Route path="clearances" element={<Manager.Clearances />} />
60         <Route path="timeline" element={<Manager.Timeline />} />
61         <Route path="help" element={<HelpPage role="Manager" />} />
62         <Route path="*" element={<Navigate to="" replace />} />
63       </Route>
64
65       <Route
66         path="/it"

```

```

67     element={role === 'it' ? <ItLayout session={session} /> : <Navigate to={`/${role}`} replace />}
68   >
69     <Route index element={<It.Dashboard />} />
70     <Route path="deprovisioning" element={<It.Deprovisioning />} />
71     <Route path="asset-recovery" element={<It.AssetRecovery />} />
72     <Route path="access-reviews" element={<It.AccessReviews />} />
73     <Route path="approvals" element={<It.Approvals />} />
74     <Route path="audit-log" element={<It.AuditLog />} />
75     <Route path="help" element={<HelpPage role="IT" />} />
76     <Route path="*" element={<Navigate to="" replace />} />
77   </Route>
78
79   <Route
80     path="/hr"
81     element={role === 'hr' ? <HrLayout session={session} /> : <Navigate to={`/${role}`} replace />}
82   >
83     <Route index element={<Hr.Dashboard />} />
84     <Route path="all-exits" element={<Hr.AllExits />} />
85     <Route path="escalations" element={<Hr.Escalations />} />
86     <Route path="risk-and-compliance" element={<Hr.RiskAndCompliance />} />
87     <Route path="exit-interviews" element={<Hr.ExitInterviews />} />
88     <Route path="trends" element={<Hr.Trends />} />
89     <Route path="clearances" element={<Hr.Clearances />} />
90     <Route path="policy-audit" element={<Hr.PolicyAudit />} />
91     <Route path="workflow-optimization" element={<Hr.WorkflowOptimization />} />
92     <Route path="reports" element={<Hr.Reports />} />
93     <Route path="agent-activity" element={<Hr.AgentActivity />} />
94     <Route path="settings" element={<Hr.Settings />} />
95     <Route path="*" element={<Navigate to="" replace />} />
96   </Route>
97
98   <Route
99     path="/finance"
100    element={role === 'finance' ? <FinanceLayout session={session} /> : <Navigate to={`/${role}`} replace />}
101  >
102    <Route index element={<Finance.Dashboard />} />
103    <Route path="help" element={<HelpPage role="Finance" />} />
104    <Route path="*" element={<Navigate to="" replace />} />
105  </Route>
106
107  <Route path="*" element={<Navigate to={`/${role}`} replace />} />
108 </Routes>

```

109)

Lines 20–28: The component receives the database-derived role and the Auth session.

Lines 31–50: Employee resignation is a separate route; employee child paths render inside EmployeeLayout.

Lines 52–103: Manager, IT, HR, and Finance each get their own guarded nested route tree.

Lines 105–109: Unknown paths redirect to the current role root.

src/lib/supabase.js

Lines 1–9 · Shared browser client

```

1 // — What this file does —————
2 // Creates and exports the single shared Supabase client used by all React
3 // components. It uses the anon (public) key, which is safe to ship in the
4 // browser because Row Level Security (RLS) in the database ensures each
5 // user can only read or write the rows they are allowed to see.
6 // —————
7 import { createClient } from '@supabase/supabase-js'
8
9 // VITE_ prefix means Vite will embed these values into the browser bundle.
```

Lines 1–4: Import the SDK and create one exported client from Vite environment variables.

Lines 5–9: The comments state the trust model: the anon key is public; RLS controls data access; the service key must not be used here.

9.2 Employee actions

src/routes/employee/EmployeePages.jsx

Lines 166–182 · Policy question and forward action

```

166 async function handleAsk() {
167   if (!question.trim() || asking) return
168   setAsking(true)
169   setAskResult(null)
170   setForwardResult(null)
171   const { data: result, error } = await supabase.functions.invoke('ask', { body: { question } })
172   setAsking(false)
173   setAskResult(error ? { error: error.message } : result)
174 }
175
176 async function handleForward() {
177   setForwarding(true)
178   setForwardResult(null)
179   const { data: result, error } = await supabase.functions.invoke('forward-to-hr', { body: { question } })
180   setForwarding(false)
181   setForwardResult(error ? { error: error.message } : result)
182 }
```

Lines 166–174: Empty or duplicate submissions are ignored; `ask` receives only the question and its result/error becomes component state.

Lines 176–182: Forwarding is a second explicit Edge Function call; it does not happen automatically on refusal.

src/routes/employee/EmployeePages.jsx

Lines 595–630 · Upload and OCR trigger

```

595  async function handleUpload(docType, file) {
596    setUploading((u) => ({ ...u, [docType]: true }))
597    setErrors((e) => ({ ...e, [docType]: '' }))
598    const ext = file.name.includes('.') ? file.name.split('.').pop() : 'dat'
599    const path = `${exitCase.id}/${docType}-${Date.now()}.${ext}`
600    const { error: uploadError } = await supabase.storage.from('exit-documents').upload(path, file)
601    if (uploadError) {
602      setUploading((u) => ({ ...u, [docType]: false }))
603      setErrors((e) => ({ ...e, [docType]: uploadError.message }))
604      return
605    }
606    const { data: inserted, error: insertError } = await supabase
607      .from('case_documents')
608      .insert({ case_id: exitCase.id, doc_type: docType, file_path: path })
609      .select('id')
610      .single()
611    if (insertError) {
612      setUploading((u) => ({ ...u, [docType]: false }))
613      setErrors((e) => ({ ...e, [docType]: insertError.message }))
614      return
615    }
616    // Trigger real OCR validation: agents/service.py is a local-only bridge
617    // (see ExitInterview's /submit-exit-interview call above). Non-fatal if
618    // it's not running -- the row just stays 'submitted' until it is.
619    try {
620      await fetch('http://localhost:8787/validate-document', {
621        method: 'POST',
622        headers: { 'Content-Type': 'application/json' },
623        body: JSON.stringify({ case_id: exitCase.id, document_id: inserted.id }),
624      })
625    } catch {
626      // agent service unreachable – non-fatal, see comment above
627    }
628    setUploading((u) => ({ ...u, [docType]: false }))
629    await load()
630  }

```

Lines 595–605: Build a case-scoped Storage path and upload the bytes; abort on upload failure.

Lines 606–615: Insert metadata and capture the new document id.

Lines 616–629: Best-effort POST triggers Python OCR/compliance; failure leaves the row submitted and the UI reloads.

src/routes/employee/EmployeePages.jsx

Lines 719–751 · Submit exit interview

```

719  async function handleSubmit(e) {
720      e.preventDefault()
721      if (busy || !reason || !wouldRecommend) return
722      setBusy(true)
723      setError('')
724      const { error } = await supabase.from('exit_interviews').insert({
725          case_id: exitCase.id,
726          reason_for_leaving: reason,
727          feedback,
728          would_recommend: wouldRecommend === 'yes',
729          comments,
730      })
731      if (error) {
732          setBusy(false)
733          setError(error.message)
734          return
735      }
736      // Trigger the Exit-Interview agent: agents/service.py is a local-only
737      // bridge (see Resignation's /activate-exit call above). If it's not
738      // running, the row is still written -- HR's analysis just won't have
739      // populated yet.
740      try {
741          await fetch('http://localhost:8787/submit-exit-interview', {
742              method: 'POST',
743              headers: { 'Content-Type': 'application/json' },
744              body: JSON.stringify({ case_id: exitCase.id }),
745          })
746      } catch {
747          // agent service unreachable – non-fatal, see comment above
748      }
749      setBusy(false)
750      setStatus('submitted')
751  }

```

Lines 719–730: Validate required fields and insert the employee-authored answers under RLS.

Lines 731–735: A database error remains visible in the form.

Lines 736–751: Best-effort service call asks the Python agent to enrich the same row; raw submission remains even if service is unavailable.

src/routes/employee/EmployeePages.jsx

Lines 832–860 · Submit resignation and activate

```

832  async function handleSubmit(e) {
833    e.preventDefault()
834    if (!lastDay || busy) return
835    setBusy(true)
836    setError('')
837    const { data, error } = await supabase.functions.invoke('submit-resignation', {
838      body: { last_working_day: lastDay, reason },
839    })
840    if (error || data?.error) {
841      setBusy(false)
842      setError(data?.error ?? error.message)
843      return
844    }
845    // Trigger the exit pipeline: agents/service.py is a local-only bridge
846    // (the submit-resignation Edge Function runs in Supabase's cloud and
847    // can't reach it). If it's not running, the case still exists – the
848    // checklist/manager email just won't have fired yet.
849    try {
850      await fetch('http://localhost:8787/activate-exit', {
851        method: 'POST',
852        headers: { 'Content-Type': 'application/json' },
853        body: JSON.stringify({ case_id: data.case.id }),
854      })
855    } catch {
856      // agent service unreachable – non-fatal, see comment above
857    }
858    setBusy(false)
859    navigate('/employee', { replace: true })
860  }

```

Lines 832–844: Invoke the authenticated Edge Function; stop on either transport or returned application error.

Lines 845–857: The returned case id is sent to the local activation service. This follow-up is intentionally non-fatal.

Lines 858–860: Clear busy state and navigate to the dashboard.

9.3 Human gates

src/routes/manager/ManagerPages.jsx

Lines 66–105 · Approve KT and trigger advancement

```

66  function useApprove(reload) {
67    const [actioning, setActioning] = useState({})
68    async function approveTask(task) {
69      const taskId = task.id

```

```

70   setActioning((a) => ({ ...a, [taskId]: 'pending' }))

71   const { error } = await supabase.from('exit_tasks').update({ status: 'done' }).eq('id', taskId)

72   if (error) {

73     setActioning((a) => ({ ...a, [taskId]: error.message }))

74     return

75   }

76   // Approving KT is the manager gate's approved branch, and on the browser
77   // path nothing else takes it: /manager-approve advances the case to IT
78   // (agents/service.py), exactly as supervisor.py's manager_gate --approved-->
79   // it edge does. The service re-checks in the database that every KT task is
80   // done before it advances, so this is a trigger, not the decision. Every
81   // approve site here is manager-stage today ('Review' and 'Sign' both act on
82   // stage='manager' rows); the guard keeps it that way if a finance-stage row
83   // is ever routed through this hook, which 0008 would also permit.
84   // Non-fatal if the service isn't running: the approval above already stuck,
85   // same posture as ItPages.jsx's /execute-deprovisioning call.
86   if (task.stage === 'manager') {

87     try {

88       await fetch('http://localhost:8787/manager-approve', {

89         method: 'POST',

90         headers: { 'Content-Type': 'application/json' },

91         body: JSON.stringify({ case_id: task.case_id }),

92       })

93     } catch {

94       // agent service unreachable – non-fatal, see comment above

95     }

96   }

97   await reload()

98   setActioning((a) => {

99     const next = { ...a }

100     delete next[taskId]

101     return next

102   })

103 }

104 return [actioning, approveTask]

105 }

```

Lines 66–75: Directly update the selected task to done; RLS decides whether that update is allowed.

Lines 76–96: For manager-stage tasks, call `/manager-approve`; the service rechecks the complete task set.

Lines 97–105: Reload and remove the action state after the write.

src/routes/manager/ManagerPages.jsx

Lines 112–140 · Reject KT

```

112 function useReject(reload) {

```

```

113   const [rejecting, setRejecting] = useState({})
114   async function rejectTask(taskId, caseId) {
115     const reason = window.prompt('Reason for rejecting this KT plan (required):')?.trim()
116     if (!reason) return
117     setRejecting((r) => ({ ...r, [taskId]: 'pending' }))
118     try {
119       const res = await fetch('http://localhost:8787/reject-manager-task', {
120         method: 'POST',
121         headers: { 'Content-Type': 'application/json' },
122         body: JSON.stringify({ case_id: caseId, task_id: taskId, reason }),
123       })
124       const data = await res.json().catch(() => ({}))
125       if (!res.ok || data.error) {
126         setRejecting((r) => ({ ...r, [taskId]: data.error || `Reject failed (HTTP ${res.status})` }))
127         return
128       }
129     } catch (err) {
130       setRejecting((r) => ({ ...r, [taskId]: `Agent service unreachable -- ${err.message}` }))
131       return
132     }
133     await reload()
134     setRejecting((r) => {
135       const next = { ...r }
136       delete next[taskId]
137       return next
138     })
139   }
140   return [rejecting, rejectTask]

```

Lines 112–117: Require a human-entered reason before calling the service.

Lines 118–132: Unlike best-effort follow-ups, rejection has no prior database write, so HTTP/service failures are shown.

Lines 133–140: Reload only after confirmed success.

src/routes/it/ltPages.jsx

Lines 49–78 · Approve and execute IT task

```

49   function useApprove(reload) {
50     const [actioning, setActioning] = useState({})
51     async function approveTask(taskId, caseId) {
52       setActioning((a) => ({ ...a, [taskId]: 'pending' }))
53       const { error } = await supabase.from('exit_tasks').update({ status: 'done' }).eq('id', taskId)
54       if (error) {
55         setActioning((a) => ({ ...a, [taskId]: error.message }))
56         return

```

```

57   }

58   // Trigger real execute/verify/audit (agent #18): agents/service.py is a
59   // local-only bridge (see EmployeePages.jsx's /validate-document call).
60   // Non-fatal if it's not running -- the approval itself already stuck.
61   try {
62     await fetch('http://localhost:8787/execute-deprovisioning', {
63       method: 'POST',
64       headers: { 'Content-Type': 'application/json' },
65       body: JSON.stringify({ case_id: caseId }),
66     })
67   } catch {
68     // agent service unreachable — non-fatal, see comment above
69   }
70   await reload()
71   setActioning((a) => {
72     const next = { ...a }
73     delete next[taskId]
74     return next
75   })
76 }
77 return [actioning, approveTask]
78 }

```

Lines 49–57: The human approval is the direct RLS-protected status update.

Lines 58–69: The service then executes/verifies every approved unaudited IT task for the case.

Lines 70–78: Reload shows task and verification state.

src/routes/finance/FinancePages.jsx

Lines 20–51 · Settle dues and recompute agents

```

20 function useSettle(reload) {
21   const [actioning, setActioning] = useState({})
22   async function settle(caseId, duesNote) {
23     setActioning((a) => ({ ...a, [caseId]: 'pending' }))
24     const { error } = await supabase.rpc('finance_mark_dues_settled', {
25       p_case_id: caseId,
26       p_dues_note: duesNote || null,
27     })
28     if (error) {
29       setActioning((a) => ({ ...a, [caseId]: error.message }))
30       return
31     }
32     // Trigger the finance/compliance re-check: agents/service.py is the
33     // same local-only bridge as every other page's fetch call. Non-fatal
34     // if it's not running -- the settlement itself already stuck.

```

```

35     try {
36         await fetch('http://localhost:8787/finance-settle-check', {
37             method: 'POST',
38             headers: { 'Content-Type': 'application/json' },
39             body: JSON.stringify({ case_id: caseId }),
40         })
41     } catch {
42         // agent service unreachable – non-fatal, see comment above
43     }
44     await reload()
45     setActioning((a) => {
46         const next = { ...a }
47         delete next[caseId]
48         return next
49     })
50 }
51 return [actioning, settle]

```

Lines 20–31: Call the security-definer finance RPC instead of directly updating the hidden base case row.

Lines 32–43: Best-effort Python follow-up recalculates finance/compliance/risk.

Lines 44–51: Reload and clear action state.

src/routes/hr/HrPages.jsx

Lines 312–343 · Resolve or reroute escalation

```

312 function useEscalationAction(reload) {
313     const [acting, setActing] = useState({})
314     async function act(task, action, actorName) {
315         setActing((a) => ({ ...a, [task.id]: 'pending' }))
316         const { data, error } = await supabase
317             .from('exit_tasks')
318             .update({ escalation_state: action })
319             .eq('id', task.id)
320             .eq('escalation_state', 'open')
321             .select()
322         if (error || !data?.length) {
323             setActing((a) => ({ ...a, [task.id]: error?.message || 'Already handled -- refresh to see current status' }))
324             return
325         }
326         try {
327             await fetch('http://localhost:8787/escalation-audit', {
328                 method: 'POST',
329                 headers: { 'Content-Type': 'application/json' },
330                 body: JSON.stringify({ case_id: task.case_id, task_id: task.id, action, actor_name: actorName }),

```

```

331     })
332   } catch {
333     // audit trail only -- the real transition above already succeeded
334   }
335   await reload()
336   setActing((a) => {
337     const next = { ...a }
338     delete next[task.id]
339     return next
340   })
341 }
342 return [acting, act]
343 }

```

Lines 312–325: Update only an open escalation and request the returned row; no row means the transition lost its race or RLS rejected it.

Lines 326–334: Send a non-fatal audit event after the authoritative database transition.

Lines 335–343: Reload and clear UI state.

src/routes/hr/HrPages.jsx

Lines 510–544 · Issue relieving letter

```

510 const RELIEVING_LETTER_STAGES = ['hr', 'manager', 'it', 'finance']
511 function readyForRelievingLetter(c, tasks) {
512   if (!c.finance_cleared || c.relieving_letter_issued) return false
513   const relevant = tasks.filter((t) => t.case_id === c.id && RELIEVING_LETTER_STAGES.includes(t.stage))
514   const stagesPresent = new Set(relevant.map((t) => t.stage))
515   return stagesPresent.size === RELIEVING_LETTER_STAGES.length && relevant.every((t) => t.status === 'done')
516 }
517
518 function useIssueRelievingLetter(userId, reload) {
519   const [actioning, setActioning] = useState({})
520   async function issue(caseId) {
521     setActioning((a) => ({ ...a, [caseId]: 'pending' }))
522     const { error } = await supabase
523       .from('exit_cases')
524       .update({
525         relieving_letter_issued: true,
526         issued_at: new Date().toISOString(),
527         issued_by: userId,
528         status: 'completed',
529       })
530       .eq('id', caseId)
531     if (error) {
532       setActioning((a) => ({ ...a, [caseId]: error.message }))

```

```

533     return
534   }
535   try {
536     await fetch('http://localhost:8787/issue-relieving-letter', {
537       method: 'POST',
538       headers: { 'Content-Type': 'application/json' },
539       body: JSON.stringify({ case_id: caseId }),
540     })
541   } catch {
542     // agent service unreachable -- non-fatal, same posture as other calls
543   }
544   await reload()

```

Lines 510–516: The UI preview requires finance clearance, no prior issue, all four stages present, and every relevant task done.

Lines 518–534: HR writes issued metadata and completed status; PostgreSQL repeats/enforces the gate.

Lines 535–544: Sending the notice is a non-fatal service follow-up.

9.4 Edge Functions

supabase/functions/submit-resignation/index.ts

Lines 49–82 · Authenticate and load employee profile

```

49 Deno.serve(async (req) => {
50   if (req.method === 'OPTIONS') return new Response(null, { headers: CORS_HEADERS })
51   if (req.method !== 'POST') return new Response('Method not allowed', { status: 405, headers: CORS_HEADERS })
52
53   try {
54     const { last_working_day, reason } = await req.json()
55     if (!last_working_day || typeof last_working_day !== 'string') {
56       return json({ error: 'last_working_day is required' }, 400)
57     }
58
59     // Read the Authorization header the browser sent (contains the session JWT token).
60     const authHeader = req.headers.get('Authorization')
61     if (!authHeader) return json({ error: 'not signed in' }, 401)
62
63     // Create a second Supabase client that uses the caller's own JWT.
64     // This lets Supabase verify who is making the request.
65     // We use the anon key + the user's JWT (not the service key) so that Supabase
66     // can identify the logged-in user via their session token.
67     const authed = createClient(SUPABASE_URL, SUPABASE_ANON_KEY, {
68       global: { headers: { Authorization: authHeader } },
69       auth: { autoRefreshToken: false, persistSession: false },
70     })
71     const { data: { user } } = await authed.auth.getUser()

```

```

72   if (!user) return json({ error: 'not signed in' }, 401)
73
74   // Use the service-role (admin) client to look up the employee's profile.
75   // We need the service key here because profiles has RLS: an employee can only
76   // read their own row, but we also need to read manager/HR profiles below.
77   const { data: profile } = await admin
78     .from('profiles')
79     .select('full_name, email, employee_id, department, role')
80     .eq('id', user.id)
81     .single()
82   if (!profile || profile.role !== 'employee') return json({ error: 'not an employee' }, 403)

```

Lines 49–57: Accept OPTIONS/POST, parse the form, validate last_working_day.

Lines 59–72: Require the Authorization header and validate the JWT through `getUser`.

Lines 74–82: Load identity with the admin client, but reject non-employee callers.

supabase/functions/submit-resignation/index.ts

Lines 84–114 · Idempotent case creation

```

84   // maybeSingle() returns null (not an error) if the employee has no case yet.
85   // single() would throw an error if 0 rows were found -- wrong behaviour here.
86   const { data: existing } = await admin
87     .from('exit_cases')
88     .select('id')
89     .eq('employee_id', profile.employee_id)
90     .maybeSingle()
91   // If they already resigned, return the existing case instead of creating a duplicate.
92   if (existing) return json({ ok: true, case: existing })
93
94   const { data: manager } = await admin.from('profiles').select('id').eq('role', 'manager').limit(1).single()
95   const { data: hr } = await admin.from('profiles').select('id').eq('role', 'hr').limit(1).single()
96
97   const { data: created, error } = await admin
98     .from('exit_cases')
99     .insert({
100       employee_id: profile.employee_id,
101       employee_name: profile.full_name,
102       email: profile.email,
103       department: profile.department,
104       role_title: DEPARTMENT_TITLES[profile.department] ?? 'Specialist',
105       manager_id: manager?.id ?? null,
106       hr_id: hr?.id ?? null,
107       last_working_day,
108       resignation_reason: typeof reason === 'string' && reason.trim() ? reason.trim() : null,

```

```

109     })
110     .select()
111     .single()
112     if (error) throw new Error(error.message)
113
114     return json({ ok: true, case: created })

```

Lines 84–92: Return an existing case for the employee, preventing duplicate resignations.

Lines 94–95: Select the first seeded manager and HR profile.

Lines 97–111: Insert server-derived identity plus client-provided date/reason and return the row.

supabase/functions/ask/index.ts

Lines 82–98 · Question embedding

```

82 // embed() converts a text string into a vector of 1536 numbers.
83 // Semantically similar texts produce similar vectors, so we can find
84 // policy chunks that are "close in meaning" to the user's question.
85 // This is called "text embedding" -- the foundation of RAG search.
86 async function embed(input: string): Promise<number[]> {
87   const res = await fetch(`${PORTKEY_BASE_URL}/v1/embeddings`, {
88     method: 'POST',
89     headers: {
90       Authorization: `Bearer ${PORTKEY_API_KEY}`,
91       'Content-Type': 'application/json',
92     },
93     body: JSON.stringify({ model: PORTKEY_VIRTUAL_KEY, input }),
94   })
95   if (!res.ok) throw new Error(`embeddings ${res.status}: ${await res.text()}`)
96   const json = await res.json()
97   return json.data[0].embedding
98 }

```

Lines 82–94: POST the input to the configured embeddings route with bearer authentication.

Lines 95–98: Fail on non-2xx response and return the first vector.

supabase/functions/ask/index.ts

Lines 122–178 · Parse LLM output and call model

```

122 function parseReply(raw: string): { answer: string; sections: string[]; scope: Scope | null } {
123   const block = raw.match(/\{[^\s\S]*\}/)?.[0]
124   const asStrings = (value: unknown): string[] =>
125     Array.isArray(value) ? value.filter((s): s is string => typeof s === 'string') : []
126
127   if (block) {
128     try {
129       const parsed = JSON.parse(block)
130       if (typeof parsed?.answer === 'string') {

```

```

131     return {
132         answer: stripCitationLine(parsed.answer),
133         sections: asStrings(parsed.sections),
134         scope: asScope(parsed.scope),
135     }
136 }
137 } catch {
138     // Almost-JSON (usually an unescaped newline). Pull the fields out by hand
139     // -- showing the employee raw JSON scaffolding would be worse.
140     const answer = block.match(/"answer"\s*:\s*"((?:[^\"]|\\.)*)"/)?.[1]
141     if (answer !== undefined) {
142         const list = block.match(/"sections"\s*:\s*\[[^\s\S]*\]/)?.[1] ?? ''
143         return {
144             answer: stripCitationLine(jsonUnescape(answer)),
145             sections: [...list.matchAll(/"((?:[^\"]|\\.)*)"/g)].map((m) => jsonUnescape(m[1])),
146             scope: asScope(block.match(/"scope"\s*:\s*"(\w+)"/)?.[1]),
147         }
148     }
149 }
150 }
151 return { answer: stripCitationLine(raw), sections: [], scope: null }
152 }
153
154 async function askModel(context: string, question: string): Promise<string> {
155     const res = await fetch(`${ANTHROPIC_BASE_URL}/v1/messages`, {
156         method: 'POST',
157         headers: {
158             Authorization: `Bearer ${ANTHROPIC_API_KEY}`,
159             'anthropic-version': '2023-06-01',
160             'Content-Type': 'application/json',
161         },
162         body: JSON.stringify({
163             model: ANTHROPIC_MODEL,
164             // Headroom for the JSON envelope, and for a reasoning model that spends
165             // output budget before emitting text (empty content on max_tokens).
166             max_tokens: 700,
167             system: SYSTEM_PROMPT,
168             messages: [
169                 { role: 'user', content: `Context:\n${context}\n\nQuestion: ${question}` },
170             ],
171         }),
172     })

```

```

173   if (!res.ok) throw new Error(`messages ${res.status}: ${await res.text()}`)
174   const json = await res.json()
175   const textBlock = json.content.find((block: { type: string }) => block.type === 'text')
176   if (!textBlock) throw new Error(`messages: no text block in content: ${JSON.stringify(json.content)}`)
177   return textBlock.text
178 }

```

Lines 122–151: Parse the requested JSON, tolerate fenced or almost-JSON output, and fall back to raw answer text.

Lines 154–177: Send system prompt plus labelled context/question to the configured message endpoint and extract the text block.

supabase/functions/forward-to-hr/index.ts

Lines 43–71 · Authenticate explicit forwarding

```

43 Deno.serve(async (req) => {
44   if (req.method === 'OPTIONS') return new Response(null, { headers: CORS_HEADERS })
45   if (req.method !== 'POST') return new Response('Method not allowed', { status: 405, headers: CORS_HEADERS })
46
47   try {
48     const { question } = await req.json()
49     if (!question || typeof question !== 'string') {
50       return json({ error: 'question is required' }, 400)
51     }
52
53     // Extract the JWT from the Authorization header — never trust identity from the request body.
54     const authHeader = req.headers.get('Authorization')
55     if (!authHeader) return json({ error: 'not signed in' }, 401)
56
57     // Anon-key client seeded with the caller's own JWT: getUser() validates the token
58     // server-side and returns the real authenticated user, not what the client claims.
59     const authed = createClient(SUPABASE_URL, SUPABASE_ANON_KEY, {
60       global: { headers: { Authorization: authHeader } },
61       auth: { autoRefreshToken: false, persistSession: false },
62     })
63
64     const { data: { user } } = await authed.auth.getUser()
65     if (!user) return json({ error: 'not signed in' }, 401)
66
67     const { data: profile } = await authed
68       .from('profiles')
69       .select('full_name, employee_id')
70       .eq('id', user.id)
71       .single()

```

Lines 43–51: Accept only POST and validate a string question.

Lines 53–65: Read and validate caller identity from the JWT, not the request body.

Lines 67–71: Read the caller's own safe profile through the authenticated client.

supabase/functions/forward-to-hr/index.ts

Lines 73–130 · Send or durably log

```

73     try {
74         const client = new SMTPClient({
75             connection: {
76                 hostname: 'smtp.gmail.com',
77                 port: 465,
78                 tls: true,
79                 auth: { username: EMAIL_SENDER, password: EMAIL_APP_PASSWORD },
80             },
81         })
82
83         await withTimeout(
84             client.send({
85                 from: EMAIL_SENDER,
86                 to: HR_FORWARD_EMAIL,
87                 subject: `ExitAI: unanswered question from ${profile?.full_name ?? user.email}`,
88                 content: [
89                     `ExitAI couldn't answer this employee's question from the exit policy docs.`,
90                     ``,
91                     `Employee: ${profile?.full_name ?? '(unknown name)'} (${profile?.employee_id ?? user.id})`,
92                     `Question: ${question}`,
93                 ].join('\n'),
94             }),
95             SMTP_TIMEOUT_MS,
96         )
97         await client.close()
98
99         return json({ ok: true })
100     } catch (sendErr) {
101         // SMTP is flaky/slow in ways that are never the employee's fault --
102         // never surface a bare dispatch error or hang. Record the question so
103         // HR still gets it (via the escalation queue, not email), and tell the
104         // employee the truth instead of a generic failure.
105         const { data: recentCase } = profile?.employee_id
106             ? await admin
107                 .from('exit_cases')
108                 .select('id')
109                 .eq('employee_id', profile.employee_id)
110                 .order('created_at', { ascending: false })
111                 .limit(1)

```

```

112         .maybeSingle()
113         : { data: null }
114
115     if (recentCase) {
116         await admin.from('agent_runs').insert({
117             case_id: recentCase.id,
118             stage: 'forward_to_hr',
119             agent: 'forward_to_hr',
120             status: 'delivery_failed',
121             detail: `Email to HR failed/timed out -- question logged: ${question}`,
122             metadata: { employee_id: profile?.employee_id ?? null, error: String(sendErr) },
123         })
124     }
125
126     return json({
127         ok: true,
128         delayed: true,
129         message: "Your question has been logged for HR. Email delivery may be delayed, but HR will still see it.",
130     })

```

Lines 73–99: Open Gmail SMTP, send within an eight-second timeout, close, and return success.

Lines 100–124: On SMTP failure, find the employee's recent case and store the question/error in agent_runs.

Lines 126–130: Return a delayed-success message because HR can still see the logged item.

9.5 Python service

agents/service.py

Lines 67–105 · Activation and interview entry points

```

67 def activate_case(case_id: str) -> dict:
68     case = db.table("exit_cases").select("*").eq("id", case_id).single().execute().data
69     if not case:
70         return {"error": "case not found"}
71
72     checklist_result = hr_agent.generate_checklist(case_id)
73
74     db.table("exit_cases").update({"status": "in_progress"}).eq("id", case_id).execute()
75     log_db("update", "exit_cases", rows=1, detail="status -> in_progress")
76
77     manager_notice = email_drafting_agent.resignation_notice(case)
78     return {"ok": True, "checklist": checklist_result, "manager_notice": manager_notice}
79
80
81 def issue_relieving_letter(case_id: str) -> dict:

```

```

82     # The frontend already flipped relieving_letter_issued/status via its own
83     # anon-key UPDATE (RLS-gated by exit_cases_hr_relieving_letter, 0017) --
84     # this just sends the completion-style notice. Non-fatal if this service
85     # isn't running, same posture as every other call in this file.
86     case = db.table("exit_cases").select("*").eq("id", case_id).single().execute().data
87     if not case:
88         return {"error": "case not found"}
89     result = email_drafting_agent.relieving_letter_notice(case)
90     return {"ok": True, "notice": result}
91
92
93 def submit_exit_interview(case_id: str) -> dict:
94     # The frontend already INSERTed the raw fields directly (anon key + RLS,
95     # per CLAUDE.md) -- this just reads them back with the service key and
96     # runs the Exit-Interview agent (exit_intel_agent.run_per_case), which
97     # UPDATES the same row's summary/sentiment/themes/rehire_* in place.
98     row = db.table("exit_interviews").select("*").eq("case_id", case_id).single().execute().data
99     if not row:
100         return {"error": "no exit interview found for case"}
101     lines = [f"Reason for leaving: {row.get('reason_for_leaving') or '(not given)'}"]
102     if row.get("would_recommend") is not None:
103         lines.append(f"Would recommend the company to a friend: {'Yes' if row['would_recommend'] else 'No'}")
104     if row.get("feedback"):
105         lines.append(f"Feedback: {row['feedback']}")

```

Lines 67–80: Activation loads the case, generates an idempotent checklist, marks it in progress, and sends a notice.

Lines 83–92: Relieving endpoint only sends; the HR browser update already changed the case.

Lines 95–105: Interview endpoint reads raw fields and formats a transcript for exit intelligence.

agents/service.py

Lines 113–155 · Document, IT, and finance events

```

113     # The frontend already uploaded the file to Storage and INSERTed the
114     # case_documents row itself (anon key + RLS, per CLAUDE.md) -- this just
115     # OCR-validates that one row with the service key. Non-fatal if this
116     # service isn't running, same posture as every other call in this file.
117     if not document_id:
118         return {"error": "document_id required"}
119     result = doc_collection.validate_one(case_id, document_id)
120     # A validated NDA/Asset Return Form can satisfy compliance_agent's
121     # matching item directly (DOC_TYPES) -- re-run its existing, idempotent
122     # single-case check so the compliance dashboard reflects it without a
123     # manual run_case. Scoped to this one case, not the full pipeline.
124     compliance = compliance_agent.run_for_case(case_id)
125     return {"ok": True, "validation": result, "compliance": compliance}

```

```

126
127
128 def execute_deprovisioning(case_id: str) -> dict:
129     # The frontend already flipped the task's status to 'done' via its own
130     # anon-key UPDATE (exit_tasks_it_update RLS, ItPages.jsx's Approve button)
131     # -- this just runs Execute/Verify/Audit (agent #18) with the service key.
132     # Non-fatal if this service isn't running, same posture as every other
133     # call in this file.
134     result = it_deprovisioning_agent.execute_approved_tasks(case_id)
135     # IT approval can satisfy compliance_agent's "access revoked" item --
136     # re-run the same existing, idempotent single-case check.
137     compliance = compliance_agent.run_for_case(case_id)
138     return {"ok": True, "executed": result, "compliance": compliance}
139
140
141 def finance_settle_check(case_id: str) -> dict:
142     # The frontend already set exit_cases.finance_cleared via its own RLS-
143     # scoped RPC (finance_mark_dues_settled, 0014/0027) -- unlike every other
144     # trigger in this file there is no existing service.py hook for this
145     # action, so this is a new one. finance_agent.check_clearance flips the
146     # stage='finance' exit_tasks row to 'done' (HrPages.jsx's
147     # readyForRelievingLetter requires that row, not just the flag, to be
148     # done) and compliance_agent.run_for_case updates its own
149     # finance_approval item. Both are idempotent, single-case re-checks --
150     # not the full pipeline.
151     #
152     # risk_agent.run_for_case is also just an entry-point call, same shape as
153     # the two above -- this is the browser-triggered path's mirror of
154     # supervisor._assess_stage's position in the CLI graph (last node, right
155     # after finance clears). Without this, risk_score/risk_level/

```

Lines 113–126: Validate one uploaded document, then immediately recompute compliance.

Lines 129–138: Execute approved IT tasks and recompute compliance.

Lines 141–155: Recompute finance, compliance, and risk after the finance RPC.

agents/service.py

Lines 158–230 · Manager approval gate

```

158     # `run_case`. A plain UPDATE, safe to re-run.
159     finance = finance_agent.check_clearance(case_id)
160     compliance = compliance_agent.run_for_case(case_id)
161     risk = risk_agent.run_for_case(case_id)
162     return {"ok": True, "finance": finance, "compliance": compliance, "risk": risk["result"]}
163

```

```

164
165 def manager_approve(case_id: str) -> dict:
166     # The APPROVED branch of agents/hub/supervisor.py's manager gate, made callable
167     # for one case -- the mirror of reject_manager_task below, and the same
168     # "re-run one stage for one case" shape as finance_settle_check above.
169     #
170     # ManagerPages.jsx's Review button already flipped the KT task to 'done'
171     # with the anon key (0008 pins a manager to their own reports' rows and to
172     # status='done'), so this never performs the approval and never bypasses
173     # the human gate: it reads the case back with the service key and advances
174     # ONLY if every KT task is genuinely done in the database. A call for a
175     # case whose KT is still outstanding is a no-op.
176     #
177     # Two writes the browser path had no way to make, both of which the
178     # supervisor's approved branch makes:
179     #   - agent_runs(stage='manager', detail='approved') -- the only source
180     #     compliance_agent._manager_item reads for its manager_approval item;
181     #   - the stage='it' exit_tasks rows, which nothing outside supervisor_graph
182     #     generated, leaving compliance_agent._it_item on "no IT task found"
183     #     and the relieving gate permanently shut.
184     # Plain select, not .single(): .single() RAISES on 0 rows (PGRST116), so
185     # the "case not found" guard below would never be reached.
186     if not db.table("exit_cases").select("id").eq("id", case_id).execute().data:
187         return {"error": "case not found"}
188
189     manager_tasks = (
190         db.table("exit_tasks").select("id, title, status, escalation_state")
191         .eq("case_id", case_id).eq("stage", "manager").execute().data or []
192     )
193     # The escalation row is a manager-stage row too, but HR closes it through
194     # escalation_state (HrPages.jsx) and never through status -- it stays
195     # 'pending' forever, so counting it as outstanding KT work would keep the
196     # gate shut for good on any case that was ever rejected.
197     escalation_ids = {t["id"] for t in manager_tasks if (t.get("title") or "").startswith("Escalated")}
198     if any(t["id"] in escalation_ids and t.get("escalation_state") == "open" for t in manager_tasks):
199         return {"ok": True, "advanced": False, "reason": "open escalation -- HR must resolve or re-route it first"}
200
201     kt_tasks = [t for t in manager_tasks if t["id"] not in escalation_ids]
202     if not kt_tasks:
203         return {"ok": True, "advanced": False, "reason": "no KT tasks on this case yet"}
204     pending = [t for t in kt_tasks if t.get("status") != "done"]
205     if pending:

```

```

206         return {
207             "ok": True, "advanced": False,
208             "reason": f"{len(pending)} of {len(kt_tasks)} KT task(s) still awaiting manager approval",
209         }
210
211     # Exactly the row supervisor._manager_gate records, in the shape
212     # compliance_agent.items_node queries for -- written once per opening, so
213     # approving the 2nd..nth KT task doesn't stack duplicates.
214     latest = (
215         db.table("agent_runs").select("detail").eq("case_id", case_id).eq("stage", "manager")
216         .in_("detail", ["approved", "rejected"]).order("created_at", desc=True).limit(1).execute().data or []
217     )
218     if not latest or latest[0]["detail"] != "approved":
219         db.table("agent_runs").insert({"case_id": case_id, "stage": "manager", "detail": "approved"}).execute()
220         log_db("insert", "agent_runs", rows=1, detail="manager gate: approved")
221
222     # Same call supervisor._it_stage makes (the #18-traced alias for
223     # it_agent.generate_plan), already idempotent: skips when IT tasks exist.
224     it_plan = it_deprovisioning_agent.generate(case_id)
225     if not it_plan.get("skipped"):
226         db.table("agent_runs").insert(
227             {"case_id": case_id, "stage": "it", "detail": "it: deprovisioning plan done"}
228         ).execute()
229         log_db("insert", "agent_runs", rows=1, detail="it: deprovisioning plan done")
230

```

Lines 158–189: The comments define this as the browser equivalent of the approved supervisor branch; first verify case existence.

Lines 191–211: Separate escalation rows from KT rows, block on open escalation, missing KT, or pending KT.

Lines 213–223: Persist one latest approved decision when necessary.

Lines 225–230: Generate the idempotent IT plan; the following lines record IT and run compliance.

agents/service.py

Lines 233–296 · Manager rejection and escalation audit

```

233
234
235 def reject_manager_task(case_id: str, task_id: str | None, reason: str | None) -> dict:
236     # Unlike every call above, this one IS the write, not a non-fatal
237     # side-effect: exit_tasks has no INSERT policy for any role (0002/0004)
238     # and its UPDATE policies (0008) pin status to 'done', so a manager's
239     # anon-key client has no RLS path to record a rejection at all. This
240     # mirrors agents.hub.supervisor._escalate's exact insert -- not a second
241     # implementation, just that same DB write made callable for one already
242     # in-progress task instead of only from a full graph run.
243     if not task_id:

```

```

244         return {"error": "task_id required"}
245     reason = (reason or "").strip()
246     if not reason:
247         return {"error": "reason required"}
248     task = db.table("exit_tasks").select("*").eq("id", task_id).single().execute().data
249     if not task or task["case_id"] != case_id:
250         return {"error": "task not found for this case"}
251     if task["stage"] != "manager" or task["title"].startswith("Escalated"):
252         return {"error": "task is not a pending manager KT task"}
253
254     already = (
255         db.table("exit_tasks").select("id, escalation_state").eq("case_id", case_id)
256         .ilike("title", "Escalated%").execute().data
257     )
258     existing = already[0] if already else None
259
260     if existing and existing["escalation_state"] == "resolved":
261         return {"error": "escalation already resolved for this case"}
262     if existing and existing["escalation_state"] == "open":
263         return {"ok": True, "already_escalated": True}
264
265     if existing:
266         # Rerouted, and the manager rejected again: reopen the SAME
267         # escalation record (one per case) rather than inserting a duplicate.
268         db.table("exit_tasks").update(
269             {"reason": reason, "escalation_state": "open"}
270             .eq("id", existing["id"]).execute()
271             log_db("update", "exit_tasks", rows=1, detail="escalation reopened")
272     else:
273         db.table("exit_tasks").insert({
274             "case_id": case_id, "stage": "manager", "status": "pending",
275             "title": "Escalated: manager rejected KT plan -- HR review needed",
276             "reason": reason, "escalation_state": "open",
277         }).execute()
278         log_db("insert", "exit_tasks", rows=1, detail="escalation")
279
280     db.table("agent_runs").insert({
281         "case_id": case_id, "stage": "escalate",
282         "detail": f"escalated to HR, stopping short of IT/finance -- reason: {reason}",
283     }).execute()
284     log_db("insert", "agent_runs", rows=1, detail="escalated to HR, stopping short of IT/finance")
285     return {"ok": True}

```

```

286
287
288 def log_escalation_transition(case_id: str, task_id: str | None, action: str | None, actor_name: str | None) -> dict:
289     # HR's Re-route/Resolve buttons perform the real state change themselves,
290     # directly against Supabase with the anon key -- RLS (0025) restricts that
291     # UPDATE to HR and to a valid open->{rerouted,resolved} transition, so
292     # "only HR can resolve/re-route" is enforced by Postgres, not by this
293     # service. This call just appends the audit trail (who + when) to
294     # agent_runs, same spirit as every _record() call in agents.hub.supervisor --
295     # and like most calls in this file, it's non-fatal: the real transition
296     # already happened before this was called.

```

Lines 233–255: Validate required fields and ensure the target is a manager KT task for the case.

Lines 257–280: Prevent duplicate/open/resolved transitions; reopen an eligible row or insert the first escalation.

Lines 282–289: Persist an escalation activity row.

Lines 293–296: The audit endpoint begins by allowing only rerouted/resolved actions.

9.6 Shared Python infrastructure

agents/core/config.py

Lines 26–73 · Load runtime configuration

```

26 # load_dotenv reads the .env file and sets each line as an environment variable.
27 # Environment variables are how secrets (API keys, passwords) are passed to the
28 # program without being hardcoded in source code. override=True: a stale Windows User-level ANTHROPIC_BASE_URL
29 # (https://portkey.ai/, wrong host) was shadowing this file's correct value
30 # and load_dotenv() never overrides pre-existing env vars by default.
31 load_dotenv(Path(__file__).resolve().parent.parent.parent / ".env", override=True)
32
33 # os.environ["KEY"] reads a required variable -- throws a clear error if it's missing.
34 # os.environ.get("KEY") reads an optional variable and returns None if absent.
35 SUPABASE_URL = os.environ["SUPABASE_URL"]
36 SUPABASE_SERVICE_KEY = os.environ["SUPABASE_SERVICE_KEY"]
37 ANTHROPIC_BASE_URL = os.environ["ANTHROPIC_BASE_URL"]
38 ANTHROPIC_API_KEY = os.environ["ANTHROPIC_API_KEY"]
39 ANTHROPIC_MODEL = os.environ["ANTHROPIC_MODEL"]
40
41 # Phase 7 -- notifications. Same Gmail account + app password already used by
42 # supabase/functions/forward-to-hr/index.ts (there under EMAIL_SENDER/
43 # EMAIL_APP_PASSWORD as Edge Function secrets); this is the Python-side name
44 # already sitting in the root .env.
45 GMAIL_ADDRESS = os.environ["GMAIL_ADDRESS"]
46 GMAIL_APP_PASSWORD = os.environ["GMAIL_APP_PASSWORD"]
47 # Optional real-send gate. Unset -> notifications.py only logs emails, never
48 # calls SMTP. Set (to any value) -> real sends go out via Gmail SMTP, From

```

```

49 # GMAIL_ADDRESS, To each email's actual intended recipient (employee/manager/
50 # HR's own address from profiles.email) -- not redirected anywhere.

51 EMAIL_TEST_RECIPIENT = os.environ.get("EMAIL_TEST_RECIPIENT")
52

53 # Phase 8 -- KT calendar booking. CLAUDE.md's blueprint calls for a single
54 # CALENDAR_API_KEY, but Google Calendar's write API needs OAuth (same kind of
55 # swap Phase 7 made: GMAIL_ADDRESS/APP_PASSWORD instead of the generic
56 # EMAIL_API_KEY). A SINGLE shared demo account (aswindevv2005@gmail.com) is
57 # authorized ONCE via agents/oauth_setup.py against the existing "Exit Auth"
58 # Google OAuth Web client -- every KT event for every ExitAI user then lands
59 # on that one account's calendar (KT_CALENDAR_ID), no per-user Google login
60 # or calendar-sharing step. All optional -- unset -> calendar_booking.py only
61 # logs the booking.

62 GOOGLE_CLIENT_ID = os.environ.get("GOOGLE_CLIENT_ID")
63 GOOGLE_CLIENT_SECRET = os.environ.get("GOOGLE_CLIENT_SECRET")
64 GOOGLE_REDIRECT_URI = os.environ.get("GOOGLE_REDIRECT_URI", "http://localhost:8765/")
65 GOOGLE_REFRESH_TOKEN = os.environ.get("GOOGLE_REFRESH_TOKEN")
66 KT_CALENDAR_ID = os.environ.get("KT_CALENDAR_ID", "primary")
67

68 # Create the shared Supabase database client using the service-role key.
69 # The service-role key bypasses Row Level Security (RLS), giving the Python
70 # agent pipeline full read/write access to all tables -- appropriate for a
71 # backend service, never for the browser. Agents import `db` from this module
72 # and call db.table("exit_cases").select("*")... etc.

73 db: Client = create_client(SUPABASE_URL, SUPABASE_SERVICE_KEY)

```

Lines 26–29: Load the repository root .env with override enabled.

Lines 35–39: Required database and LLM values fail immediately when missing.

Lines 45–66: Email/calendar values include optional safety gates and defaults.

Line 73: Construct one service-role Supabase client shared by Python agents.

agents/core/llm.py

Lines 37–82 · Low-level LLM request

```

37 def _post(system: str, user: str, max_tokens: int) -> SimpleNamespace:
38     # Send the request to the Portkey gateway, which forwards it to Azure OpenAI.
39     # "system" tells the AI how to behave (its persona/rules).
40     # "user" is the actual question or task we're asking it to do.
41     resp = requests.post(
42         f"{ANTHROPIC_BASE_URL.rstrip('/')}/v1/messages",
43         headers={
44             "Authorization": f"Bearer {ANTHROPIC_API_KEY}",
45             "anthropic-version": "2023-06-01",
46             "Content-Type": "application/json",

```

```

47     },
48     json={
49         "model": ANTHROPIC_MODEL,
50         "max_tokens": max_tokens,
51         "system": system,
52         "messages": [{"role": "user", "content": user}],
53     },
54     timeout=60,
55 )
56 # raise_for_status() throws an exception if the server returned an error code
57 # (e.g. 401 Unauthorized, 429 Too Many Requests, 500 Server Error).
58 resp.raise_for_status()
59 body = resp.json()
60 # Gateway model has extended thinking on, so content[0] is sometimes a
61 # 'thinking' block with no 'text' key -- find the actual text block
62 # instead of assuming index 0.
63 text_block = next((b for b in body["content"] if b.get("type") == "text"), None)
64 if text_block is None:
65     # Reasoning models spend max_tokens on reasoning before emitting any
66     # text, and return content:[] rather than a partial block -- without
67     # this the caller just sees a bare StopIteration.
68     raise RuntimeError(
69         f"{ANTHROPIC_MODEL} returned no text block "
70         f"(stop_reason={body.get('stop_reason')!r}, usage={body.get('usage')}). "
71         "If stop_reason is 'max_tokens', raise max_tokens for this call."
72     )
73 # Wrapped as SimpleNamespace (not the raw dict) so trace.log_llm's
74 # attribute-based introspection (resp.usage.input_tokens, resp.content[0].text)
75 # works and the terminal trace shows real token counts, not "tokens n/a".
76 # SimpleNamespace turns a plain dict into an object with dot-notation access.
77 # trace.log_llm() reads resp.usage.input_tokens and resp.content[0].text
78 # as attributes, not as dict keys, so we wrap the raw dict in SimpleNamespace.
79 return SimpleNamespace(
80     content=[SimpleNamespace(text=text_block["text"])],
81     usage=SimpleNamespace(**body.get("usage", {})),
82 )

```

Lines 37–58: Build and send an Anthropic-compatible message request using configured gateway/model.

Lines 59–71: Raise on HTTP failure, find the text block, and report missing output explicitly.

Lines 73–82: Wrap text and usage in an attribute-style object used by tracing.

```

87 def ask_claude(system: str, user: str, max_tokens: int = 500) -> str:

```

```

88     resp = log_llm(ANTHROPIC_MODEL, lambda: _post(system, user, max_tokens))
89     return resp.content[0].text
90
91
92 # ask_claude_json: same as ask_claude, but parses the response as JSON.
93 # Use this when the system prompt instructs the AI to reply with a JSON object
94 # (e.g. {"hr_tasks": [...], "manager_tasks": [...]}). Returns a Python dict.
95 def ask_claude_json(system: str, user: str, max_tokens: int = 500) -> dict:
96     """Same as ask_claude, but parses the JSON object the prompt asked for
97     (tolerates ```-fenced output by grabbing the first {...} block)."""
98     text = ask_claude(system, user, max_tokens)
99     match = re.search(r"\{.*\}", text, re.DOTALL)
100     return json.loads(match.group(0) if match else text)
101

```

Lines 87–89: `ask_claude` times the request and returns text.

Lines 95–102: `ask_claude_json` extracts the first brace-delimited object and parses it.

agents/core/prompts.py

Lines 8–14 · Prompt loader

```

8 # _ROOT resolves to the /prompts/ folder two levels above agents/core/,
9 # i.e. the repo root / prompts. Path(__file__) is this file itself.
10 _ROOT = Path(__file__).resolve().parent.parent.parent / "prompts"
11
12
13 def load_prompt(rel_path: str) -> str:
14     # Read the prompt file (e.g. "interview/summary_system.md") as plain text.

```

Line 10: Resolve the repository prompts directory from this file location.

Lines 13–14: Read the requested UTF-8 markdown and remove only trailing newlines.

agents/core/trace.py

Lines 90–149 · Node, LLM, and database tracing

```

90 def traced_node(name: str):
91     """Decorator for a LangGraph node. Logs start/inputs, then done/output/time
92     (or the failure). Nested traced nodes indent under their caller."""
93     def decorator(fn):
94         @functools.wraps(fn)
95         def wrapper(state, *args, **kwargs):
96             if TRACE_LEVEL == "off":
97                 return fn(state, *args, **kwargs)
98
99             start = time.perf_counter()
100             _emit(_c(f"└─ {name}", "cyan") + _c(" ▶ start", "dim"))

```

```

101         token = _depth.set(_depth.get() + 1)
102
103         if TRACE_LEVEL == "full":
104
105             _emit(_c(f"inputs: {_fmt(state)}", "dim"))
106
107         try:
108             result = fn(state, *args, **kwargs)
109
110         except Exception as exc:
111             elapsed = time.perf_counter() - start
112             _depth.reset(token)
113             _emit(_c(f"⌋ {name}", "cyan")
114                   + _c(f" ✗ failed {elapsed:.2f}s "
115                       f"{type(exc).__name__}: {exc}", "red"))
116             raise
117
118         elapsed = time.perf_counter() - start
119         if TRACE_LEVEL == "full" and result is not None:
120             _emit(_c(f"output: {_fmt(result)}", "dim"))
121             _depth.reset(token)
122             _emit(_c(f"⌋ {name}", "cyan")
123                   + _c(f" ✓ done {elapsed:.2f}s", "green"))
124             return result
125
126     return wrapper
127
128     return decorator
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

```

143     """Log a database write: log_db('insert', 'exit_tasks', rows=7)."""
144     extra = f" ({rows} rows)" if rows is not None else ""
145     if detail:
146         extra += f" {detail}"
147     _emit(_c(f"↳ db {action} {table}", "yellow") + _c(extra, "dim"))
148
149

```

Lines 90–121: The decorator logs start/result/failure/timing and restores ContextVar depth on both paths.

Lines 126–138: `log_llm` times the supplied zero-argument call and logs tokens/preview.

Lines 141–149: `log_db` formats an audit line only; it never performs a DB operation.

agents/core/notifications.py

Lines 49–80 · Compose and send safely

```

49 def _compose(to_name: str | None, intro: str, lines: list[str] = (), closing: str = "") -> str:
50     # Build a plain-text email body: greeting, intro, optional bullet list,
51     # optional closing sentence, then the standard sign-off.
52     greeting = f"Hi {to_name}," if to_name else "Hi there,"
53     parts = [greeting, "", intro]
54     if lines:
55         parts += [""] + [f" - {l}" for l in lines]
56     if closing:
57         parts += ["", closing]
58     parts += ["", *SIGNOFF]
59     return "\n".join(parts)
60
61
62 @traced_node("Notification agent -- send email")
63 def _send(to: str, subject: str, body: str) -> dict:
64     # Dev-safety gate: if EMAIL_TEST_RECIPIENT is not set in .env, only log
65     # the email to the terminal – no real SMTP call happens. This prevents
66     # accidental emails during development.
67     if not EMAIL_TEST_RECIPIENT:
68         print(f"[email:dev-log] to={to}\nsubject={subject}\n{body}\n")
69         return {"sent": False, "logged": True, "to": to}
70
71     # smtplib is Python's built-in email sending library. SMTP_SSL uses port 465
72     # with TLS from the start (safer than STARTTLS on 587 for Gmail app passwords).
73     msg = MIMEText(body)
74     msg["Subject"] = subject
75     msg["From"] = f"{SENDER_DISPLAY} <{GMAIL_ADDRESS}>"
76     msg["To"] = to
77     with smtplib.SMTP_SSL("smtp.gmail.com", 465) as smtp:
78         smtp.login(GMAIL_ADDRESS, GMAIL_APP_PASSWORD)

```

```

79         smtp.sendmail(GMAIL_ADDRESS, [to], msg.as_string())
80     return {"sent": True, "to": to}

```

Lines 49–59: Build one plain-text message shape used by all templates.

Lines 62–68: Without EMAIL_TEST_RECIPIENT, print and return without SMTP.

Lines 70–80: Otherwise force the configured test recipient, create MIME headers, and send through Gmail SSL.

agents/core/calendar_booking.py

Lines 69–106 · Create and persist KT event

```

69 @traced_node("Calendar -- book KT event")
70 def _create_event(case: dict, task: dict) -> dict:
71     # Dev-safety gate: if OAuth isn't configured, log instead of booking a real event.
72     if not _configured():
73         print(f"[calendar:dev-log] would book KT event for {case['employee_name']}: "
74               f"{task['title']}\n (due {task.get('due_date', 'soon')})")
75         return {"booked": False, "logged": True}
76
77     # Invite both the employee and their manager as calendar attendees.
78     attendees = [e for e in (case.get("email"), _profile_email(case.get("manager_id"))) if e]
79     day = task.get("due_date") or date.today().isoformat()
80     # Google Calendar all-day event: start and end both use {"date": "YYYY-MM-DD"}.
81     event = {
82         "summary": f"KT: {case['employee_name']} handover -- {task['title']}",
83         "description": (
84             f"Knowledge-transfer session for {case['employee_name']}'s exit "
85             f"({case['role_title']}, {case['department']})."
86         ),
87         "start": {"date": day},
88         "end": {"date": day},
89         "attendees": [{"email": a} for a in attendees],
90     }
91     # sendUpdates="all" emails the invite to all attendees immediately.
92     created = (
93         _service().events()
94         .insert(calendarId=KT_CALENDAR_ID, body=event, sendUpdates="all")
95         .execute()
96     )
97     return {"booked": True, "event_id": created["id"]}
98
99
100 def book_kt_event(case: dict, task: dict) -> dict:
101     """Books the event and, if it has a real id, persists it onto the task."""
102     result = _create_event(case, task)

```

```

103     if result.get("event_id") and task.get("id"):
104         db.table("exit_tasks").update({"kt_event_id": result["event_id"]}).eq("id", task["id"]).execute()
105         log_db("update", "exit_tasks", rows=1, detail="kt_event_id")
106     return result

```

Lines 69–90: Build an all-day event from the case/task; return a logged-only result when OAuth is absent.

Lines 92–97: Insert through Calendar API and return the event id/link.

Lines 100–106: Public wrapper writes the returned event id back to the task.

9.7 Operational agents

agents/spokes/checklist_generator_agent.py

Lines 14–21 · Checklist compatibility wrapper

```

14 from ..core.trace import traced_node
15
16
17 @traced_node("Checklist Generator agent (#5)")
18 def generate(case_id: str, force: bool = False) -> dict:
19     # Delegate entirely to hr_agent – no new logic here. force=True re-runs
20     # even if tasks already exist (useful for testing; idempotency in hr_agent).
21     return hr_agent.generate_checklist(case_id, force=force)

```

Lines 14–21: One traced function delegates directly to `hr_agent.generate_checklist`; there is no second checklist implementation.

agents/spokes/kt_document_reviewer_agent.py

Lines 12–22 · KT reviewer compatibility wrapper

```

12 # hr_agent already has review_kt_document(); this module adds its own @traced_node
13 # wrapper so the step appears separately in the terminal trace log.
14 from . import hr_agent
15 from ..core.trace import traced_node
16
17
18 @traced_node("KT Document Reviewer agent (#10)")
19 def review(case_id: str, kt_text: str) -> dict:
20     # Delegate entirely to hr_agent – no new logic here.
21     # kt_text is the raw handover document text the employee submitted.
22     return hr_agent.review_kt_document(case_id, kt_text)

```

Lines 12–15: Import the existing HR review implementation and the trace decorator.

Lines 18–22: The named agent adds trace identity, then delegates all behavior to `hr_agent.review_kt_document`.

agents/spokes/hr_agent.py

Lines 92–139 · Generate and persist checklist

```

92 class ChecklistState(TypedDict):
93     case_id: str    # the UUID of the exit case being processed
94     case: dict      # the full exit_cases row (role, department, last_working_day, etc.)

```

```

95     result: dict    # filled in by the generate step; consumed by the persist step
96
97
98 @traced_node("HR agent -- generate checklist")
99 def _generate_checklist(state: ChecklistState) -> ChecklistState:
100     case = state["case"]
101     user = f"Role: {case['role_title']}\nDepartment: {case['department']}"
102     state["result"] = ask_claude_json(CHECKLIST_SYSTEM_PROMPT, user)
103     return state
104
105
106 @traced_node("HR agent -- persist checklist")
107 def _persist_checklist(state: ChecklistState) -> ChecklistState:
108     case_id = state["case_id"]
109     last_day = date.fromisoformat(state["case"]["last_working_day"])
110     r = state["result"]
111     hr_rows = [
112         {"case_id": case_id, "stage": "hr", "title": t, "status": "pending",
113          "due_date": (last_day - timedelta(days=3)).isoformat()}
114         for t in r.get("hr_tasks", [])
115     ]
116     kt_titles, it_owned = [], []
117     for t in r.get("manager_tasks", []):
118         (it_owned if IT_OWNED_TITLE_RE.search(t or "") else kt_titles).append(t)
119     if it_owned:
120         log_db("drop", "exit_tasks", rows=len(it_owned),
121              detail=f"IT-owned titles kept out of stage=manager: {it_owned}")
122     manager_rows = [
123         {"case_id": case_id, "stage": "manager", "title": t, "status": "pending",
124          "due_date": (last_day - timedelta(days=1)).isoformat()}
125         for t in kt_titles
126     ]
127     rows = hr_rows + manager_rows
128     inserted_manager_rows = []
129     if rows:
130         inserted = db.table("exit_tasks").insert(rows).execute().data or []
131         inserted_manager_rows = [r for r in inserted if r["stage"] == "manager"]
132     log_db("insert", "exit_tasks", rows=len(rows))
133
134     # KT scheduled: manager_rows are the KT tasks (ManagerDashboard.jsx's
135     # ktTasks = stage 'manager'). Remind employee + manager now that they
136     # have a due_date, and book a calendar event per task (Phase 8).

```

```

137     if manager_rows:
138         email_drafting_agent.kt_reminder(state["case"], manager_rows)
139     for t in inserted_manager_rows:

```

Lines 92–103: State carries case data and LLM result; generation sends role and department.

Lines 107–128: Persistence splits manager titles into IT-owned versus KT, creates HR/manager rows with stage-specific due dates.

Lines 129–139: Batch insert, trace the write, and book calendar events only for inserted manager rows.

agents/spokes/hr_agent.py

Lines 158–168 · Checklist idempotency and invocation

```

158 def generate_checklist(case_id: str, force: bool = False) -> dict:
159     """Idempotent unless force=True: skips if hr/manager tasks already exist
160     for this case (seeded or already agent-generated)."""
161     if not force:
162         existing = (
163             db.table("exit_tasks").select("id").eq("case_id", case_id).in_("stage", ["hr", "manager"]).execute().data
164         )
165         if existing:
166             return {"skipped": True, "reason": "hr/manager tasks already exist"}
167     case = db.table("exit_cases").select("*").eq("id", case_id).single().execute().data
168     return checklist_graph.invoke({"case_id": case_id, "case": case, "result": {}})

```

Lines 158–165: Unless forced, any existing HR/manager task makes generation return `skipped`.

Lines 167–168: Load the full case and invoke the compiled checklist graph.

agents/spokes/hr_agent.py

Lines 182–232 · Review KT document and persist gaps

```

182 @traced_node("HR agent -- review KT document")
183 def _review_kt(state: KtReviewState) -> KtReviewState:
184     # max_tokens is explicit here rather than llm.py's 500 default: this prompt
185     # asks for a three-key object whose `gaps` list grows with the document, and
186     # the gateway model spends output budget on reasoning before emitting any
187     # text. At 500 a real handover doc (~1.5k chars) returns content:[] with
188     # stop_reason='max_tokens' and llm._post raises, which takes down the whole
189     # supervisor run via kt_document_reviewer_agent. Measured: 500 fails, 1500 succeeds.
190     state["result"] = ask_claude_json(
191         KT_REVIEW_SYSTEM_PROMPT, f"Handover document:\n{state['kt_text']}", max_tokens=1500
192     )
193     return state
194
195
196 @traced_node("HR agent -- persist KT review")
197 def _persist_kt_review(state: KtReviewState) -> KtReviewState:
198     case_id = state["case_id"]

```

```

199     r = state["result"]
200     gaps = r.get("gaps", [])
201
202     # Idempotent per gap topic: a case can be KT-reviewed repeatedly (re-run,
203     # resubmitted handover), and re-inserting an identical title would stack
204     # duplicate to-dos on the manager's card.
205     existing = {
206         t["title"]
207         for t in (
208             db.table("exit_tasks")
209                 .select("title")
210                 .eq("case_id", case_id)
211                 .eq("stage", "manager")
212                 .execute()
213                 .data
214             or []
215         )
216     }
217     task_rows = [
218         {"case_id": case_id, "stage": "manager", "status": "pending", "title": title}
219         for title in (f"Add handover notes: {g['topic']}" for g in gaps)
220         if title not in existing
221     ]
222     if task_rows:
223         db.table("exit_tasks").insert(task_rows).execute()
224     log_db("insert", "exit_tasks", rows=len(task_rows))
225
226     db.table("kt_reviews").insert({
227         "case_id": case_id,
228         "summary": r["summary"],
229         "gaps": gaps,
230         "complete": r.get("complete", not gaps),
231     }).execute()
232     log_db("insert", "kt_reviews", rows=1)

```

Lines 182–193: Ask the model to return a structured review for raw KT text.

Lines 197–223: Read existing manager titles, deduplicate gap tasks, and insert neutral follow-ups.

Lines 226–232: Store the evaluative summary/gaps separately in kt_reviews.

agents/spokes/it_agent.py

Lines 44–78 · Generate and persist IT plan

```

44 class ItPlanState(TypedDict):
45     case_id: str      # UUID of the exit case
46     case: dict        # full exit_cases row (role_title, department, last_working_day)

```

```

47     result: dict      # AI's JSON response; filled by generate, consumed by persist
48
49
50 @traced_node("IT agent -- generate deprovisioning plan")
51 def _generate_plan(state: ItPlanState) -> ItPlanState:
52     case = state["case"]
53     # Tell the AI the employee's role and department so it can generate
54     # role-appropriate deprovisioning tasks (e.g. "Revoke GitHub access" for Engineering).
55     user = f"Role: {case['role_title']}\nDepartment: {case['department']}"
56     state["result"] = ask_claude_json(SYSTEM_PROMPT, user)
57     return state
58
59
60 @traced_node("IT agent -- persist deprovisioning plan")
61 def _persist_plan(state: ItPlanState) -> ItPlanState:
62     case_id = state["case_id"]
63     last_day = date.fromisoformat(state["case"]["last_working_day"])
64     # Build a database row for each task the AI generated.
65     # stage="it" routes these rows to the IT dashboard (ItPages.jsx reads stage='it').
66     rows = [
67         {"case_id": case_id, "stage": "it", "title": t, "status": "pending", "due_date": last_day.isoformat()}
68         for t in state["result"].get("tasks", [])
69     ]
70     if rows:
71         db.table("exit_tasks").insert(rows).execute()
72         log_db("insert", "exit_tasks", rows=len(rows))
73     return state
74
75
76 _graph = StateGraph(ItPlanState)
77 _graph.add_node("generate", _generate_plan)
78 _graph.add_node("persist", _persist_plan)

```

Lines 44–57: State includes case/result; generation sends role and department to the IT prompt.

Lines 60–72: Convert returned titles into stage=it pending tasks due on the last working day and insert them.

Lines 75–78: The two nodes form a generate→persist graph.

agents/spokes/it_deprovisioning_agent.py

Lines 59–84 · Classify and verify IT actions

```

59 def _classify_action(title: str) -> str:
60     """Pure function: task title in -> mock action-type out."""
61     # The order matters: ASSET_RE is checked first because "asset" is broader.
62     if ASSET_RE.search(title):

```

```

63         return "asset_recovery"
64     if IDENTITY_RE.search(title):
65         return "identity_sso_revoke"
66     if REPO_RE.search(title):
67         return "source_control_revoke"
68     # Default: anything else (e.g. "Remove from Slack") is a SaaS app revoke.
69     return "saas_app_revoke"
70
71
72 def _verify_execution(task: dict, execution: dict) -> dict:
73     """Pure function: independently re-derives the expected action type from
74     the task title and cross-checks it against what the adapter itself
75     reported, so a task the adapter mis-executed (wrong action, or ok=True
76     with no real effect) doesn't get waved through just because it ran."""
77     expected = _classify_action(task["title"])
78     # verified is True ONLY if the adapter said ok=True AND reported the right action type.
79     # This catches the "task marked done but wrong step actually ran" gap.
80     verified = execution.get("ok") is True and execution.get("action_type") == expected
81     return {
82         "verified": verified,
83         "expected_action_type": expected,
84         "reported_action_type": execution.get("action_type"),

```

Lines 59–69: Title keywords map to asset, identity, repository, or generic access actions.

Lines 72–84: Verification independently recalculates the expected type and requires both adapter success and a type match.

agents/spokes/it_deprovisioning_agent.py

Lines 123–167 · Execute approved tasks once

```

123 @traced_node("Automated IT Deprovisioning agent (#18) -- generate plan")
124 def generate(case_id: str, force: bool = False) -> dict:
125     return it_agent.generate_plan(case_id, force=force)
126
127
128 @traced_node("Automated IT Deprovisioning agent (#18) -- execute/verify/audit approved tasks")
129 def execute_approved_tasks(case_id: str) -> list[dict]:
130     """Execute Through Adapter -> Verify Execution -> Audit, for exit_tasks
131     the human has already approved (stage='it', status='done'). Idempotent:
132     skips any task_id that already has an agent_runs audit row for this
133     stage."""
134     # Only look at IT-stage tasks the human has already approved (status='done').
135     tasks = db.table("exit_tasks").select("id, title").eq("case_id", case_id).eq("stage", "it").eq("status",
136     "done").execute().data or []
137     if not tasks:
138         return []

```

```

138     # Idempotency guard: only a task that already VERIFIED is a real
139     # duplicate-side-effect risk. A task that previously failed/timed-out
140     # must stay retryable on the next call -- a task marked done that never
141     # actually executed must not get stuck that way forever (Phase 5
142     # Scenario C: retry/recovery, not a permanent dead end).
143     audited = db.table("agent_runs").select("status, metadata").eq("case_id", case_id).eq("stage",
"it_deprovisioning_execution").execute().data or []
144     # Build a set of task IDs that have already been successfully verified – skip those.
145     already_executed = {r["metadata"]["task_id"] for r in audited if r.get("status") == "verified" and r.get("metadata",
{}).get("task_id")}
146
147     adapter = MockITAdapter()
148     results = []
149     for task in tasks:
150         if task["id"] in already_executed:
151             continue
152         # Run the mock adapter (would call Okta/Jamf in production).
153         execution = adapter.execute(task)
154         # Independently check: did the adapter actually do the right thing?
155         verification = _verify_execution(task, execution)
156         status = "verified" if verification["verified"] else "verification_failed"
157         # Write one audit row per task – this is both the audit trail and the
158         # idempotency guard for the next call to execute_approved_tasks().
159         db.table("agent_runs").insert({
160             "case_id": case_id, "stage": "it_deprovisioning_execution", "agent": "it_deprovisioning_agent",
161             "status": status, "detail": f"{task['title']} -> {execution.get('detail')} ({status})",
162             "metadata": {"task_id": task["id"], "execution": execution, "verification": verification},
163         }).execute()
164         results.append({"task_id": task["id"], "title": task["title"], "execution": execution, "verification":
verification})
165
166     log_db("insert", "agent_runs", rows=len(results), detail="it_deprovisioning_execution")
167     return results

```

Lines 123–126: Planning wrapper delegates to the ordinary IT plan agent.

Lines 129–146: Select only done IT tasks, load prior audits, and build the set of already verified task ids.

Lines 148–167: Skip verified tasks, execute and verify remaining tasks, then insert one audit row per attempt.

agents/spokes/finance_agent.py

Lines 46–91 · Calculate finance clearance

```

46 @traced_node("Finance agent -- check clearance")
47 def _check_clearance(state: FinanceState) -> FinanceState:
48     case_id = state["case_id"]
49     # Fetch all HR, manager, and IT tasks for this case.
50     # "all(...)" returns True only if every task's status is "done".

```

```

51 other_tasks = db.table("exit_tasks").select("status, stage").eq("case_id", case_id).in_(
52     "stage", ["hr", "manager", "it"]
53 ).execute().data or []
54 stages_done = bool(other_tasks) and all(t["status"] == "done" for t in other_tasks)
55
56 case = db.table("exit_cases").select("*").eq("id", case_id).single().execute().data or {}
57 # finance_cleared is set by the Finance dashboard's "Mark dues settled" button.
58 dues_settled = bool(case.get("finance_cleared"))
59 # Both conditions must be true: all prior work done AND finance manually confirmed.
60 cleared = stages_done and dues_settled
61 reason = None if cleared else ("dues/settlement not confirmed" if stages_done else "prior stages not complete")
62 title = "Clear final settlement dues" if cleared else f"Clear final settlement dues -- blocked: {reason}"
63
64 finance_tasks = db.table("exit_tasks").select("id, status").eq("case_id", case_id).eq("stage",
"finance").execute().data or []
65 if not finance_tasks:
66     # No finance task exists yet -- create one with the right initial status.
67     db.table("exit_tasks").insert({
68         "case_id": case_id, "stage": "finance", "title": title,
69         "status": "done" if cleared else "pending",
70     }).execute()
71     log_db("insert", "exit_tasks", rows=1)
72     newly_cleared = cleared # it's a new "done" row, so this is a fresh clearance
73 else:
74     new_status = "done" if cleared else "pending"
75     # newly_cleared: True only on the first time we flip from pending -> done.
76     # If it was already "done" on every previous run, this evaluates to False
77     # and we skip the completion email -- avoiding re-sends on every re-check.
78     newly_cleared = cleared and any(t["status"] != new_status for t in finance_tasks)
79     for t in finance_tasks:
80         db.table("exit_tasks").update({"status": new_status, "title": title}).eq("id", t["id"]).execute()
81     log_db("update", "exit_tasks", rows=len(finance_tasks), detail=f"status={new_status}")
82
83 # Clearance completes: notify only on the pending -> done transition, not
84 # on every re-run once it's already settled.
85 if newly_cleared and case:
86     email_drafting_agent.completion_notice(case)
87
88 state["result"] = {"cleared": cleared, "reason": reason}
89 return state
90
91

```

Lines 46–60: Read HR/manager/IT task statuses and the case finance flag; clearance requires both.

Lines 61–77: Build a done/pending finance row and insert it when absent.

Lines 78–86: Update existing rows and detect only a new transition to done.

Lines 88–91: Send completion notice once and store result in graph state.

agents/spokes/compliance_agent.py

Lines 44–118 · Evaluate compliance signals

```

44 def evaluate(tasks: list[dict], validated_doc_types: frozenset[str] = frozenset()) -> dict:
45     """Pure function: which of the three compliance checks are satisfied. A check
46     with no matching task at all counts as missing (blocks), not N/A. A validated
47     case_documents row for the item's doc_type (see DOC_TYPES) also satisfies it."""
48     blocking = []
49     for label, pattern in CHECKS.items():
50         matches = [t for t in tasks if pattern.search(t.get("title", ""))]
51         done = [t for t in matches if t.get("status") == "done"]
52         doc_type = DOC_TYPES.get(ITEM_IDS[label])
53         if done or (doc_type and doc_type in validated_doc_types):
54             continue
55         if not matches:
56             blocking.append(f"{label}: no task found")
57         else:
58             blocking.append(f"{label}: pending")
59     return {"cleared": not blocking, "blocking_reasons": blocking}
60
61
62 # ---- item-level traceability (docs/agent_requirements.md #13 P1 upgrade) -----
63 # Every item is sourced from real, already-written data -- no new upstream
64 # gate logic. asset_return/nda/access_revoked reuse the keyword match above
65 # (now per-item instead of aggregated); manager/it/finance approval read the
66 # same rows the human-approval gates themselves already write (supervisor.py's
67 # manager_gate -> agent_runs, ItDashboard.jsx's Approve button -> exit_tasks
68 # status, the Finance dashboard's "Mark dues settled" -> exit_cases.finance_cleared).
69
70 def _keyword_item(item: str, pattern, tasks: list[dict], validated_doc_types: frozenset[str] = frozenset()) -> dict:
71     matches = [t for t in tasks if pattern.search(t.get("title", ""))]
72     done = [t for t in matches if t.get("status") == "done"]
73     if done:
74         return {"item": item, "status": "done", "source": "exit_tasks",
75                 "evidence": done[0].get("title"), "failure_reason": None}
76     doc_type = DOC_TYPES.get(item)
77     if doc_type and doc_type in validated_doc_types:
78         return {"item": item, "status": "done", "source": "case_documents",
79                 "evidence": f"{doc_type} validated", "failure_reason": None}

```

```

80     if not matches:
81         return {"item": item, "status": "missing", "source": "exit_tasks",
82                "evidence": None, "failure_reason": "no matching task found"}
83     return {"item": item, "status": "pending", "source": "exit_tasks",
84            "evidence": matches[0].get("title"), "failure_reason": "task not yet done"}
85
86
87 def _manager_item(manager_detail: str | None) -> dict:
88     if manager_detail is None:
89         return {"item": "manager_approval", "status": "missing", "source": "agent_runs.manager_gate",
90                "evidence": None, "failure_reason": "manager gate not yet recorded"}
91     if manager_detail == "approved":
92         return {"item": "manager_approval", "status": "done", "source": "agent_runs.manager_gate",
93                "evidence": manager_detail, "failure_reason": None}
94     return {"item": "manager_approval", "status": "pending", "source": "agent_runs.manager_gate",
95            "evidence": manager_detail, "failure_reason": f"manager gate recorded: {manager_detail}"}
96
97
98 def _it_item(it_tasks: list[dict]) -> dict:
99     if not it_tasks:
100         return {"item": "it_approval", "status": "missing", "source": "exit_tasks.stage=it",
101                "evidence": None, "failure_reason": "no IT task found"}
102     if all(t.get("status") == "done" for t in it_tasks):
103         return {"item": "it_approval", "status": "done", "source": "exit_tasks.stage=it",
104                "evidence": f"{len(it_tasks)} task(s) done", "failure_reason": None}
105     return {"item": "it_approval", "status": "pending", "source": "exit_tasks.stage=it",
106            "evidence": f"{len(it_tasks)} task(s)", "failure_reason": "IT tasks not all done (awaiting IT approval)"}
107
108
109 def _finance_item(finance_cleared: bool) -> dict:
110     if finance_cleared:
111         return {"item": "finance_approval", "status": "done", "source": "exit_cases.finance_cleared",
112                "evidence": "finance_cleared=true", "failure_reason": None}
113     return {"item": "finance_approval", "status": "pending", "source": "exit_cases.finance_cleared",
114            "evidence": "finance_cleared=false", "failure_reason": "dues/settlement not confirmed"}
115
116
117 # CHECKS keys are the human-readable labels used in evaluate()'s
118 # blocking_reasons text (untouched); item ids here are the normalized,

```

Lines 44–67: Legacy three-item evaluation uses matching done tasks or validated matching documents.

Lines 70–84: Item-level helper distinguishes missing evidence from incomplete evidence.

Lines 87–118: Manager approval, IT completion, and finance clearance each use explicit signals rather than keyword matching.

agents/spokes/compliance_agent.py

Lines 151–215 · Persist summary and item checks

```

151 @traced_node("Compliance Verification -- check")
152 def _check_node(state: ComplianceState) -> ComplianceState:
153     case_id = state["case_id"]
154     tasks = db.table("exit_tasks").select("title, status, stage").eq("case_id", case_id).execute().data or []
155     # exclude the compliance-stage summary task itself -- its own title (e.g.
156     # "...NDA: no task found...") is written by _persist_node and would
157     # otherwise keyword-match against the very checks below, self-poisoning
158     # every future run. Same filter _items_node already applies.
159     keyword_tasks = [t for t in tasks if t.get("stage") != "compliance"]
160     state["result"] = evaluate(keyword_tasks, _validated_doc_types(case_id))
161     return state
162
163
164 @traced_node("Compliance Verification -- persist")
165 def _persist_node(state: ComplianceState) -> ComplianceState:
166     case_id = state["case_id"]
167     result = state["result"]
168     status = "done" if result["cleared"] else "pending"
169     title = (
170         "Compliance verified -- cleared for final clearance" if result["cleared"]
171         else "Final clearance blocked: " + "; ".join(result["blocking_reasons"])
172     )
173     existing = db.table("exit_tasks").select("id").eq("case_id", case_id).eq("stage", "compliance").execute().data
174     if existing:
175         db.table("exit_tasks").update({"title": title, "status": status}).eq("id", existing[0]["id"]).execute()
176         log_db("update", "exit_tasks", rows=1, detail=title)
177     else:
178         db.table("exit_tasks").insert(
179             {"case_id": case_id, "stage": "compliance", "title": title, "status": status}
180         ).execute()
181         log_db("insert", "exit_tasks", rows=1, detail=title)
182     return state
183
184
185 @traced_node("Compliance Verification -- item-level trace (#13)")
186 def _items_node(state: ComplianceState) -> ComplianceState:
187     case_id = state["case_id"]
188     tasks = db.table("exit_tasks").select("title, status, stage").eq("case_id", case_id).execute().data or []
189     manager_rows = (

```

```

190         db.table("agent_runs").select("detail, created_at").eq("case_id", case_id).eq("stage", "manager")
191         .in_("detail", ["approved", "rejected"]).order("created_at", desc=True).limit(1).execute().data or []
192     )
193     manager_detail = manager_rows[0]["detail"] if manager_rows else None
194     it_tasks = [t for t in tasks if t.get("stage") == "it"]
195     case = db.table("exit_cases").select("finance_cleared").eq("id", case_id).single().execute().data or {}
196     finance_cleared = bool(case.get("finance_cleared"))
197
198     # exclude the compliance-stage summary task itself: its own title (e.g.
199     # "...NDA: no task found...") is written by _persist_node and would
200     # otherwise keyword-match against the very NDA/asset/access checks below.
201     keyword_tasks = [t for t in tasks if t.get("stage") != "compliance"]
202     items = evaluate_items(keyword_tasks, manager_detail, it_tasks, finance_cleared, _validated_doc_types(case_id))
203     now = datetime.now(timezone.utc).isoformat()
204     for item in items:
205         db.table("compliance_checks").upsert({
206             "case_id": case_id,
207             "item": item["item"],
208             "status": item["status"],
209             "source": item["source"],
210             "evidence": item["evidence"],
211             "failure_reason": item["failure_reason"],
212             "checked_at": now,
213         }, on_conflict="case_id,item").execute()
214     done = sum(1 for i in items if i["status"] == "done")
215     log_db("upsert", "compliance_checks", rows=len(items), detail=f"{done}/{len(items)} done")

```

Lines 151–161: Read non-compliance tasks and validated document types, then compute the summary result.

Lines 164–181: Update or insert exactly one stage=compliance summary task.

Lines 185–215: Read latest manager decision and finance flag, evaluate five explicit items, and upsert each compliance_checks row.

agents/spokes/risk_agent.py

Lines 56–100 · Compute risk deterministically

```

56 def _tenure_risk(created_at: str | None) -> float:
57     """Shorter tenure -> higher risk (less institutional investment to lose).
58     Proxy for the missing hire_date column -- see module docstring."""
59     if not created_at:
60         return 0.5
61     created = datetime.fromisoformat(created_at.replace("Z", "+00:00"))
62     days = (datetime.now(timezone.utc) - created).days
63     if days < 90:
64         return 0.9
65     if days < 365:

```

```

66         return 0.6

67     return 0.3

68

69

70 def _tasks_risk(total: int, done: int) -> float:

71     if total == 0:

72         return 0.0

73     return round((total - done) / total, 2)

74

75

76 def score(case: dict, profile: dict | None, interview: dict | None, tasks: list[dict]) -> dict:

77     # Each factor returns a value between 0.0 and 1.0 (higher = more risky).

78     tenure = _tenure_risk(profile.get("created_at") if profile else None)

79     role = DEPARTMENT_CRITICALITY.get(case.get("department"), 0.5) # default 0.5 for unknown depts

80     sentiment = SENTIMENT_RISK.get((interview or {}).get("sentiment"), 0.5) # default neutral

81     total_tasks = len(tasks)

82     done_tasks = sum(1 for t in tasks if t.get("status") == "done")

83     tasks_risk = _tasks_risk(total_tasks, done_tasks)

84

85     # Weighted average of all four factors. Each weight scales its factor's

86     # contribution so the total is always between 0.0 and 1.0.

87     risk_score = round(

88         tenure * WEIGHTS["tenure"]

89         + role * WEIGHTS["role"]

90         + sentiment * WEIGHTS["sentiment"]

91         + tasks_risk * WEIGHTS["tasks"],

92         3,

93     )

94     # Bucket the continuous score into three labels for the HR dashboard.

95     risk_level = "high" if risk_score >= 0.66 else "medium" if risk_score >= 0.4 else "low"

96     # Prefer the AI's explicit rehire verdict (from the exit interview) if it exists;

97     # otherwise infer: a high-risk departure is unlikely to be a good rehire candidate.

98     rehire_eligible = (interview or {}).get("rehire_eligible", risk_level != "high")

99

100     return {"risk_score": risk_score, "risk_level": risk_level, "rehire_eligible": rehire_eligible}

```

Lines 56–68: Parse profile creation time as a tenure proxy and return a banded factor.

Lines 70–73: Incomplete-task ratio supplies the task factor.

Lines 76–96: Look up the other factors, apply fixed weights, assign risk band, and choose rehire eligibility.

Lines 97–100: Return only the three values persisted on exit_cases.

agents/spokes/risk_agent.py

Lines 108–147 · Load, score, and persist risk

```

108 @traced_node("Compliance & Risk -- score")

```

```

109 def _score_node(state: RiskState) -> RiskState:
110     case_id = state["case_id"]
111     case = db.table("exit_cases").select("*").eq("id", case_id).single().execute().data
112     profile = (
113         db.table("profiles").select("created_at").eq("employee_id", case["employee_id"]).limit(1).execute().data
114         if case.get("employee_id")
115         else None
116     )
117     profile = profile[0] if profile else None
118     interview = db.table("exit_interviews").select("sentiment, rehire_eligible").eq("case_id", case_id).execute().data
119     tasks = db.table("exit_tasks").select("status").eq("case_id", case_id).execute().data or []
120     state["result"] = score(case, profile, interview[0] if interview else None, tasks)
121     return state
122
123
124 @traced_node("Compliance & Risk -- persist")
125 def _persist_node(state: RiskState) -> RiskState:
126     db.table("exit_cases").update(state["result"]).eq("id", state["case_id"]).execute()
127     log_db("update", "exit_cases", rows=1, detail=str(state["result"]))
128     # Same "own agent_runs row" pattern smart_routing/rehire_agent already use,
129     # so risk scoring has an audit trail too, not just the exit_cases write.
130     r = state["result"]
131     detail = f"score={r['risk_score']} level={r['risk_level']} rehire_eligible={r['rehire_eligible']}"
132     db.table("agent_runs").insert({"case_id": state["case_id"], "stage": "risk", "detail": detail}).execute()
133     log_db("insert", "agent_runs", rows=1, detail=detail)
134     return state
135
136
137 _graph = StateGraph(RiskState)
138 _graph.add_node("score", _score_node)
139 _graph.add_node("persist", _persist_node)
140 _graph.set_entry_point("score")
141 _graph.add_edge("score", "persist")
142 _graph.set_finish_point("persist")
143 risk_graph = _graph.compile()
144
145
146 def run_for_case(case_id: str) -> dict:
147     return risk_graph.invoke({"case_id": case_id, "result": {}})

```

Lines 108–121: Load case, employee profile creation time, interview signal, and all task statuses; call pure `score`.

Lines 124–133: Update the case and insert a risk audit row.

Lines 146–147: Public entry invokes the two-node graph.

agents/spokes/exit_intel_agent.py

Lines 38–86 · Analyze and persist one interview

```

38 class PerCaseState(TypedDict):
39     case_id: str
40     interview_text: str
41     result: dict          # filled by _analyze; consumed by _persist
42
43
44 @traced_node("Exit-Interview Intelligence -- analyze")
45 def _analyze(state: PerCaseState) -> PerCaseState:
46     # ask_claude_json() sends the transcript to the LLM and parses the JSON reply.
47     # The LLM returns summary/sentiment/themes/rehire_eligible/rehire_reason.
48     state["result"] = ask_claude_json(SYSTEM_PROMPT, f"Transcript:\n{state['interview_text']}")
49     return state
50
51
52 @traced_node("Exit-Interview Intelligence -- persist")
53 def _persist(state: PerCaseState) -> PerCaseState:
54     r = state["result"]
55     row = {
56         "case_id": state["case_id"],
57         "summary": r["summary"],
58         "sentiment": r["sentiment"],
59         "themes": r["themes"],
60         "rehire_eligible": r["rehire_eligible"],
61         "rehire_reason": r["rehire_reason"],
62     }
63     # Idempotency: if an interview row already exists for this case, update it;
64     # otherwise insert a new one. Prevents duplicate rows on re-submission.
65     existing = db.table("exit_interviews").select("id").eq("case_id", state["case_id"]).execute()
66     if existing.data:
67         db.table("exit_interviews").update(row).eq("case_id", state["case_id"]).execute()
68         log_db("update", "exit_interviews", rows=1)
69     else:
70         db.table("exit_interviews").insert(row).execute()
71         log_db("insert", "exit_interviews", rows=1)
72     return state
73
74
75 # Two-node per-case subgraph: analyze (LLM) -> persist (DB write).
76 _per_case = StateGraph(PerCaseState)

```

```

77 _per_case.add_node("analyze", _analyze)
78 _per_case.add_node("persist", _persist)
79 _per_case.set_entry_point("analyze")
80 _per_case.add_edge("analyze", "persist")
81 _per_case.set_finish_point("persist")
82 per_case_graph = _per_case.compile()
83
84
85 def run_per_case(case_id: str, interview_text: str) -> dict:
86     return per_case_graph.invoke({"case_id": case_id, "interview_text": interview_text, "result": {}})

```

Lines 38–49: Graph state holds transcript/result; the analyze node sends the transcript to the structured prompt.

Lines 52–72: Map model output to database fields and update an existing interview or insert one.

Lines 77–86: Compile analyze→persist and expose `run_per_case`.

agents/spokes/doc_collection.py

Lines 83–115 · Document requirements and content rules

```

83 def required_documents(department: str | None) -> list[str]:
84     """Pure function: department in -> required doc-type list out."""
85     return BASE_REQUIRED_DOCS + DEPT_EXTRA_DOCS.get(department or "", [])
86
87
88 def classify_documents(required: list[str], submitted_rows: list[dict]) -> dict:
89     """Pure function: required list + case_documents rows in -> required vs
90     submitted vs missing out. A row in ANY status (submitted/validated/
91     rejected) counts as "submitted" -- rejected means uploaded-but-invalid,
92     not missing."""
93     submitted_types = {r["doc_type"] for r in submitted_rows}
94     return {
95         "required": required,
96         "submitted": [d for d in required if d in submitted_types],
97         "missing": [d for d in required if d not in submitted_types],
98     }
99
100
101 def validate_content(doc_type: str, text: str, employee_name: str | None = None) -> dict:
102     """Pure function: doc type + OCR'd text (+ the case's employee name, when
103     identity is applicable) in -> validation verdict out. No I/O, no LLM --
104     everything here is a substring/regex check against real text. Caller must
105     confirm doc_type is actually required for this case first -- an
106     unrecognized doc_type has no VALIDATION_RULES entry, so it would
107     otherwise auto-pass with missing=[]."""
108     text_lower = text.lower()

```

```

109     matched, missing = [], []
110     for group in VALIDATION_RULES.get(doc_type, []):
111         (matched if any(p in text_lower for p in group) else missing).append(group[0])
112     if employee_name:
113         (matched if employee_name.lower() in text_lower else missing).append("employee identity")
114     (matched if DATE_RE.search(text) else missing).append("date")
115     return {"ok": not missing, "matched": matched, "missing": missing}

```

Lines 83–86: Required document types are base plus department-specific extras.

Lines 88–98: Classification computes submitted and missing sets without storing missing rows.

Lines 101–115: Validation requires at least one keyword in every rule group, then optional employee-name and date checks.

agents/spokes/doc_collection.py

Lines 156–214 · OCR one stored document

```

156 def _required_type_verdict(doc_type: str, department: str | None, text: str, employee_name: str | None = None) -> dict:
157     """Pure function: guards validate_content with the one check it can't do
158     itself -- is doc_type even required for this department? An unrecognized
159     doc_type has no VALIDATION_RULES entry, so validate_content alone would
160     silently auto-pass it (missing=[])."""
161     if doc_type not in required_documents(department):
162         return {"ok": False, "matched": [], "missing": ["not a required document type for this case"]}
163     return validate_content(doc_type, text, employee_name)
164
165
166 def _validate_row(case_id: str, department: str | None, employee_name: str | None, row: dict) -> dict:
167     """Real work behind one case_documents row: download the uploaded object
168     from Storage, OCR it, then _required_type_verdict. Shared by the batch
169     _validate node and the single-document validate_one() entry point below."""
170     required = row["doc_type"] in required_documents(department)
171     if required:
172         file_bytes = db.storage.from_("exit-documents").download(row["file_path"])
173         text = pytesseract.image_to_string(Image.open(io.BytesIO(file_bytes)))
174         print(f"[doc_collection] OCR output for {row['doc_type']} ({row['file_path']}): \n{text}")
175     else:
176         text = ""
177     verdict = _required_type_verdict(row["doc_type"], department, text, employee_name)
178     new_status = "validated" if verdict["ok"] else "rejected"
179     detail = f"matched={verdict['matched']} missing={verdict['missing']}"
180     db.table("case_documents").update({"status": new_status, "validation_detail": detail}).eq("id", row["id"]).execute()
181     db.table("agent_runs").insert({
182         "case_id": case_id, "stage": "doc_collection",
183         "detail": f"OCR-validated {row['doc_type']} -> {new_status} ({detail})",
184     }).execute()
185     return {"doc_type": row["doc_type"], "status": new_status, "ocr_text": text, **verdict}

```

```

186
187
188 @traced_node("Document Collection -- OCR validate uploads")
189 def _validate(state: DocState) -> DocState:
190     validations = [
191         _validate_row(state["case_id"], state["_case"].get("department"), state["_case"].get("employee_name"), row)
192         for row in state["_submitted_rows"] if row["status"] == "submitted"
193     ]
194     log_db("update", "case_documents", rows=len(validations), detail="OCR validation")
195     state["validations"] = validations
196     return state
197
198
199 def validate_one(case_id: str, document_id: str) -> dict:
200     """Validate a single just-uploaded row without re-running the whole
201     graph -- _remind unconditionally re-sends a reminder email for every
202     still-missing doc type on each run(case_id) call, so re-running the full
203     graph after every upload would spam reminders. Called by
204     agents.service's /validate-document route right after an employee
205     upload."""
206     case = db.table("exit_cases").select("id, department, employee_name").eq("id", case_id).single().execute().data
207     row = db.table("case_documents").select("*").eq("id", document_id).eq("case_id", case_id).single().execute().data
208     return _validate_row(case_id, case.get("department"), case.get("employee_name"), row)
209
210
211 _graph = StateGraph(DocState)
212 _graph.add_node("gather", _gather)
213 _graph.add_node("remind", _remind)
214 _graph.add_node("validate", _validate)

```

Lines 156–163: Company Asset Declaration uses department-aware asset/identity/date checks; other types use shared rules.

Lines 166–185: Download bytes, OCR through Pillow/Tesseract, evaluate, update status/detail, and audit.

Lines 199–214: `validate_one` binds the document to the case before running the same row validator.

agents/spokes/smart_routing.py

Lines 42–67 · Choose an approver

```

42 def select_approver(candidates: list[dict], department: str | None) -> dict:
43     """Pure function: ordered candidate profiles (earliest-created first) +
44     a case department in -> routing decision out. No I/O, no LLM."""
45     if not candidates:
46         return {"approver": None, "is_delegate": False, "all_ooo": False, "reason": "no profiles hold this stage's role"}
47
48     # Prefer a candidate from the same department as the exiting employee (if any).

```

```

49     # In practice, hr/manager/it profiles have no department set yet, so this
50     # always falls through to the full candidate list – see module docstring.
51     dept_matches = [c for c in candidates if department and c.get("department") == department]
52     pool = dept_matches or candidates
53     # Filter out anyone marked out_of_office; remaining list is in created_at order.
54     available = [c for c in pool if not c.get("out_of_office")]
55
56     if available:
57         chosen = available[0]
58         # pool[0] is the "primary" (earliest-created); if we picked someone else, it's a delegate.
59         is_delegate = chosen["id"] != pool[0]["id"]
60         reason = (f"primary ({pool[0]['full_name']}) is out_of_office -> routed to delegate"
61                  if is_delegate else "primary approver available")
62         return {"approver": chosen, "is_delegate": is_delegate, "all_ooo": False, "reason": reason}
63
64     # All candidates are OOO – default to the primary rather than blocking the pipeline.
65     chosen = pool[0]
66     return {"approver": chosen, "is_delegate": False, "all_ooo": True,
67           "reason": "every candidate is out_of_office -> defaulted to primary (no one actually available)"}

```

Lines 42–49: Empty candidates produce a non-crashing “no approver” decision.

Lines 51–57: Prefer department matches, then the first available person; delegate means the first choice was skipped.

Lines 59–67: If everyone is out, choose the primary but explicitly flag all_ooo.

agents/spokes/multi_system_clearance.py

Lines 166–201 · Fetch and consolidate system statuses

```

166 @traced_node("Multi-System Clearance -- query stand-in adapters")
167 def _fetch(state: ClearanceState) -> ClearanceState:
168     case_id = state["case_id"]
169     state["_responses"] = {
170         "hrms": HRMSAdapter().check_clearance(case_id),
171         "itam": ITAMAdapter().check_clearance(case_id),
172         "finance": FinanceAdapter().check_clearance(case_id),
173     }
174     state["_doc_rows"] = db.table("case_documents").select("doc_type, status").eq("case_id", case_id).execute().data or []
175     return state
176
177
178 @traced_node("Multi-System Clearance -- consolidate")
179 def _consolidate(state: ClearanceState) -> ClearanceState:
180     status = consolidate_status(state["_responses"], state["_doc_rows"])
181     state["status"] = status
182     detail = f"overall={status['overall']} it={status['it_asset_management']['status']} " \
183             f"hrms={status['hrms']['status']} finance={status['finance']['status']}"

```

```

184     db.table("agent_runs").insert({"case_id": state["case_id"], "stage": "multi_system_clearance", "detail":
        detail}).execute()

185     log_db("insert", "agent_runs", rows=1, detail=detail)

186     return state

187

188

189 _graph = StateGraph(ClearanceState)
190 _graph.add_node("fetch", _fetch)
191 _graph.add_node("consolidate", _consolidate)
192 _graph.set_entry_point("fetch")
193 _graph.add_edge("fetch", "consolidate")
194 _graph.set_finish_point("consolidate")

195 multi_system_clearance_graph = _graph.compile()

196

197

198 def consolidate(case_id: str) -> dict:
199     """The tool entry point supervisor.py calls from the compliance stage node."""
200     result = multi_system_clearance_graph.invoke({"case_id": case_id, "_responses": {}, "_doc_rows": [], "status": {}})
201     return result["status"]

```

Lines 166–175: Call each stand-in adapter and separately load all case document rows.

Lines 178–187: Consolidate, build a detailed audit line, and insert agent_runs.

Lines 198–201: Public function invokes the graph and returns only its status object.

agents/spokes/email_drafting_agent.py

Lines 42–83 · Audit every email outcome

```

42 def _record_email(case_id: str | None, template: str, result: dict | None, error: str | None = None) -> None:
43     # Skip the DB write if we have no case_id to attach the record to.
44     if not case_id:
45         return
46     outcomes = _outcomes(result) if result is not None else []
47     status = "failed" if error else _status(outcomes)
48     metadata = {
49         "template": template,
50         "recipients": [o.get("to") for o in outcomes if o.get("to")],
51         "outcomes": outcomes,
52     }
53     if error:
54         metadata["error"] = error
55     # One audit row per email attempt, regardless of whether it sent or logged.
56     db.table("agent_runs").insert({
57         "case_id": case_id,
58         "stage": "email_drafting",
59         "agent": "email_drafting_agent",

```

```

60         "status": status,
61         "detail": f"{template} -> {status}",
62         "metadata": metadata,
63     }).execute()
64     log_db("insert", "agent_runs", rows=1, detail=f"email_drafting:{template}:{status}")
65
66
67 def _drafted(template: str, case_id: str | None, fn, *args, **kwargs) -> dict:
68     """Never lets a send failure propagate: a real SMTP/network error here
69     must not abort the rest of whatever pipeline called us (e.g. finance_agent's
70     caller still needs to run the risk/assess stage; sla_escalation's caller
71     still needs to notify the *other* breaches in this batch). The failure is
72     still fully recorded to agent_runs -- this only stops the exception, not
73     the audit trail (Phase 5 Scenario E: 'workflow state remains consistent')."""
74     try:
75         result = fn(*args, **kwargs)
76     except Exception as exc:
77         # Capture the error message and write it to agent_runs, then return a safe
78         # failure dict so callers get a consistent shape regardless of what happened.
79         error = f"{type(exc).__name__}: {exc}"
80         _record_email(case_id, template, None, error=error)
81         return {"sent": False, "logged": False, "to": None, "error": error}
82     _record_email(case_id, template, result)
83     return result

```

Lines 42–64: Normalize recipients/outcomes into agent_runs metadata and insert one email-drafting audit.

Lines 67–80: Call the notification function, catch any exception, audit failure, and return without raising.

Lines 82–83: The following exported template functions all use this wrapper.

agents/spokes/rehire_agent.py

Lines 30–61 · Standalone rehire recommendation

```

30 def _assess(rehire_eligible: bool, risk_level: str, rehire_reason: str | None) -> dict:
31     """Pure function: real inputs in -> eligibility/confidence/evidence out.
32     Confidence is deterministic, not guessed: "high" when an actual interview
33     rationale exists to back the eligibility flag, else "medium" (eligibility
34     alone, from risk_agent's risk_level fallback)."""
35     evidence = [f"risk_agent.risk_level={risk_level}"]
36     if rehire_reason:
37         evidence.append("exit_interviews.rehire_reason")
38     return {
39         "rehire_eligible": rehire_eligible,
40         "rehire_reason": rehire_reason,
41         "confidence": "high" if rehire_reason else "medium",

```

```

42     "evidence": evidence,
43 }
44
45
46 @traced_node("Intelligent Rehire Assessment agent (#21)")
47 def assess(case_id: str) -> dict:
48     result = risk_agent.run_for_case(case_id)["result"]
49     interview = db.table("exit_interviews").select("rehire_reason").eq("case_id", case_id).execute().data
50     rehire_reason = interview[0]["rehire_reason"] if interview else None
51     assessment = _assess(result["rehire_eligible"], result["risk_level"], rehire_reason)
52     db.table("agent_runs").insert({
53         "case_id": case_id,
54         "stage": "rehire_assessment",
55         "agent": "rehire_agent",
56         "status": "done",
57         "detail": f"rehire_eligible={assessment['rehire_eligible']} confidence={assessment['confidence']}",
58         "metadata": assessment,
59     }).execute()
60     log_db("insert", "agent_runs", rows=1, detail="rehire_assessment")
61     return assessment

```

Lines 30–43: Eligibility false wins; otherwise high risk triggers review, and remaining cases are recommended.

Lines 46–61: Read existing computed fields/reason, build assessment, insert audit, return it. This does not update exit_cases.

9.8 Analytics and capstone runners

agents/analytics/analytics_agent.py

Lines 34–84 · Aggregate, narrate, persist

```

34 @traced_node("Analytics -- aggregate")
35 def _aggregate(state: AnalyticsState) -> AnalyticsState:
36     # Read all cases and tasks from the database in two queries.
37     # No LLM here — arithmetic and counting only (CLAUDE.md non-negotiable).
38     cases = db.table("exit_cases").select("department, status, risk_level").execute().data or []
39     tasks = db.table("exit_tasks").select("case_id, status, stage").execute().data or []
40
41     # Counter is a dict subclass that counts occurrences of each value.
42     by_department = Counter(c["department"] for c in cases)
43     by_status = Counter(c["status"] for c in cases)
44     by_risk = Counter(c["risk_level"] for c in cases if c.get("risk_level"))
45
46     # Which pipeline stages currently have the most stuck (not-done) tasks?
47     pending_by_stage = Counter(t["stage"] for t in tasks if t["status"] != "done")
48     total_tasks = len(tasks)
49     done_tasks = sum(1 for t in tasks if t["status"] == "done")

```

```

50
51     state["stats"] = {
52         "total_cases": len(cases),
53         "cases_by_department": dict(by_department),
54         "cases_by_status": dict(by_status),
55         "cases_by_risk_level": dict(by_risk),
56         "pending_tasks_by_stage": dict(pending_by_stage),
57         # None if there are no tasks at all (avoids division by zero).
58         "task_completion_rate": round(done_tasks / total_tasks, 2) if total_tasks else None,
59     }
60     return state
61
62
63 @traced_node("Analytics -- narrate")
64 def _narrate(state: AnalyticsState) -> AnalyticsState:
65     # Only NOW does the LLM appear – it turns the numbers dict into a
66     # 2-3 sentence HR-friendly insight summary. It reads stats, never computes them.
67     state["narrative"] = ask_claude(SYSTEM_PROMPT, f"Stats:\n{state['stats']}")
68     return state
69
70
71 @traced_node("Analytics -- persist")
72 def _persist(state: AnalyticsState) -> AnalyticsState:
73     # Write both the raw stats dict and the LLM narrative to analytics_insights,
74     # so the HR dashboard can display the text and the raw numbers separately.
75     db.table("analytics_insights").insert({
76         "narrative": state["narrative"],
77         "stats": state["stats"],
78         "agent_type": "dashboard_insights",
79     }).execute()
80     log_db("insert", "analytics_insights", rows=1)
81     return state
82
83
84 # Build the three-node LangGraph subgraph: aggregate -> narrate -> persist.

```

Lines 34–59: Load cases/tasks and calculate counts in Python.

Lines 63–68: The model receives already-computed stats and writes only narrative.

Lines 71–84: Persist narrative, raw stats, and agent_type.

agents/analytics/sla_escalation.py

Lines 39–72 · Find real SLA breaches

```

39 def find_breaches(tasks: list[dict], cases: dict[str, dict], profiles: dict[str, dict], today: date) -> list[dict]:

```

```

40     """Pure function: pending tasks + their cases/blockers in -> breaches
41     (>=THRESHOLD_DAYS overdue) out, each naming the blocker, duration and
42     impact. No I/O, no LLM -- everything here is arithmetic/lookups."""
43     breaches = []
44     for t in tasks:
45         # Skip tasks that aren't pending, or that have no due date to compare against.
46         if t.get("status") != "pending" or not t.get("due_date"):
47             continue
48         # due_date may come back as a Python date object or a string -- handle both.
49         due = t["due_date"] if isinstance(t["due_date"], date) else date.fromisoformat(t["due_date"])
50         days_overdue = (today - due).days
51         if days_overdue < THRESHOLD_DAYS:
52             continue
53         case = cases.get(t["case_id"])
54         if not case:
55             continue
56         stage = t["stage"]
57         # Name the actual person blocking this stage if one exists in profiles.
58         if stage == "hr":
59             blocker = profiles.get(case.get("hr_id"), {}).get("full_name") or "HR"
60         elif stage == "manager":
61             blocker = profiles.get(case.get("manager_id"), {}).get("full_name") or "the manager"
62         else:
63             # IT / finance / compliance don't have a per-case named assignee yet.
64             blocker = STAGE_TEAM.get(stage, f"the {stage} team")
65         breaches.append({
66             "task_id": t["id"], "case_id": t["case_id"], "stage": stage, "title": t["title"],
67             "days_overdue": days_overdue, "blocker": blocker,
68             "employee_name": case.get("employee_name"),
69             "impact": f"blocks final clearance for {case.get('employee_name')} ({case.get('department', 'n/a')})",
70             "hr_id": case.get("hr_id"), "manager_id": case.get("manager_id"),
71         })
72     return breaches

```

Lines 39–50: Ignore done/missing/not-old-enough tasks and normalize due dates.

Lines 52–72: Resolve case/owner names and emit structured breach objects containing downstream impact.

agents/analytics/workflow_optimizer.py

Lines 64–91 · Reuse breaches for optimization report

```

64 @traced_node("Exit Workflow Optimizer -- aggregate (reuses sla_escalation.find_breaches)")
65 def _aggregate(state: OptimizerState) -> OptimizerState:
66     # Reuse find_breaches() -- same overdue arithmetic already tested by sla_escalation.
67     tasks = db.table("exit_tasks").select("id, case_id, stage, title, status, due_date").eq("status",
        "pending").execute().data or []

```

```

68     cases = {c["id"]: c for c in db.table("exit_cases").select("id, employee_name, department, hr_id,
manager_id").execute().data or []}

69     profiles = {p["id"]: p for p in db.table("profiles").select("id, full_name").execute().data or []}

70     breaches = find_breaches(tasks, cases, profiles, date.today())

71     state["stats"] = bottleneck_stats(breaches)

72     return state

73

74

75 @traced_node("Exit Workflow Optimizer -- narrate")

76 def _narrate(state: OptimizerState) -> OptimizerState:

77     # LLM turns the bottleneck numbers into a recommendation sentence.

78     state["narrative"] = ask_claude(SYSTEM_PROMPT, f"Stats:\n{state['stats']}")

79     return state

80

81

82 @traced_node("Exit Workflow Optimizer -- persist")

83 def _persist(state: OptimizerState) -> OptimizerState:

84     db.table("analytics_insights").insert({

85         "narrative": state["narrative"],

86         "stats": state["stats"],

87         "agent_type": "workflow_optimizer",

88     }).execute()

89     log_db("insert", "analytics_insights", rows=1)

90     return state

91

```

Lines 64–72: Load pending tasks/cases/profiles, reuse `find_breaches`, and compute bottleneck stats.

Lines 75–79: Ask the model to narrate those stats.

Lines 82–91: Store the report under workflow_optimizer agent_type.

agents/analytics/policy_auditor.py

Lines 168–220 · Gather, audit, report

```

168 @traced_node("Policy Auditor -- gather active cases")

169 def _gather(state: AuditorState) -> AuditorState:

170     cases = db.table("exit_cases").select("id, employee_name, department, hr_id, manager_id, status").execute().data or []

171     active = [c for c in cases if c.get("status") in ACTIVE_STATUSES]

172     case_ids = [c["id"] for c in active]

173

174     all_tasks = db.table("exit_tasks").select("id, case_id, stage, title, status, due_date").execute().data or []

175     tasks_by_case: dict[str, list[dict]] = {}

176     all_tasks_by_case: dict[str, list[dict]] = {}

177     for t in all_tasks:

178         if t["case_id"] in case_ids:

179             all_tasks_by_case.setdefault(t["case_id"], []).append(t)

```

```

180         if t["status"] == "pending":
181             tasks_by_case.setdefault(t["case_id"], []).append(t)
182
183     runs = db.table("agent_runs").select("case_id, stage, detail").execute().data or []
184     approvals_by_case = {
185         r["case_id"]: True for r in runs if r["stage"] == "manager" and r.get("detail") == "approved"
186     }
187
188     state["_active"] = active
189     state["_tasks_by_case"] = tasks_by_case
190     state["_all_tasks_by_case"] = all_tasks_by_case
191     state["_approvals_by_case"] = approvals_by_case
192     return state
193
194
195 @traced_node("Policy Auditor -- audit")
196 def _audit(state: AuditorState) -> AuditorState:
197     state["report"] = audit_cases(
198         state["_active"], state["_tasks_by_case"], state["_all_tasks_by_case"],
199         state["_approvals_by_case"], date.today(),
200     )
201     return state
202
203
204 @traced_node("Policy Auditor -- report")
205 def _report(state: AuditorState) -> AuditorState:
206     report = state["report"]
207     if report["breaches"]:
208         state["narrative"] = ask_claude(SYSTEM_PROMPT, f"Audit report:\n{report}")
209     else:
210         state["narrative"] = "No policy breaches found across active cases in this audit."
211     db.table("analytics_insights").insert({
212         "narrative": state["narrative"], "stats": report, "agent_type": AGENT_TYPE,
213     }).execute()
214     log_db("insert", "analytics_insights", rows=1)
215     return state
216
217
218 # Three-node graph: gather real rows, run the pure audit function, write the narrative.
219 _graph = StateGraph(AuditorState)
220 _graph.add_node("gather", _gather)

```

Lines 168–192: Load active cases, relevant tasks, and manager decision logs into per-case structures.

Lines 195–200: Run the deterministic audit function.

Lines 204–220: Narrate only when breaches exist and persist both report and counts.

agents/analytics/attrition_agent.py

Lines 79–116 · Department-level attrition signals

```

79 @traced_node("Predictive Attrition -- gather signals")
80 def _gather(state: AttritionState) -> AttritionState:
81     # Fetch all three data sources: departed cases (with risk scores),
82     # trend alerts (rising exit-interview themes), and current employees.
83     state["cases"] = db.table("exit_cases").select("department, risk_score").execute().data or []
84     state["alerts"] = db.table("trend_alerts").select("department, severity, theme").execute().data or []
85     state["employees"] = (
86         db.table("profiles").select("employee_id, full_name, department").eq("role", "employee").execute().data or []
87     )
88     return state
89
90
91 @traced_node("Predictive Attrition -- identify at-risk")
92 def _identify(state: AttritionState) -> AttritionState:
93     # Pure function: no I/O, no LLM – all arithmetic and set comparisons.
94     state["signals"] = identify_at_risk(state["cases"], state["alerts"], state["employees"])
95     return state
96
97
98 @traced_node("Predictive Attrition -- narrate & persist")
99 def _narrate_and_persist(state: AttritionState) -> AttritionState:
100     signals = state["signals"]
101     if signals["at_risk_departments"]:
102         # LLM writes the retention-intervention narrative from the flagged depts list.
103         state["narrative"] = ask_claude(SYSTEM_PROMPT, f"At-risk departments:\n{signals['at_risk_departments']}")
104     else:
105         state["narrative"] = "No department currently shows a rising attrition signal."
106     db.table("analytics_insights").insert({
107         "narrative": state["narrative"],
108         "stats": signals,
109         "agent_type": "predictive_attrition",
110     }).execute()
111     log_db("insert", "analytics_insights", rows=1)
112     return state
113
114
115 # Three-node graph: gather -> identify -> narrate+persist.
116 _graph = StateGraph(AttritionState)

```

Lines 79–88: Load case risks, trend alerts, and current employee profiles.

Lines 91–95: Run deterministic department signal selection.

Lines 98–116: Narrate only nonempty risk signals and persist with predictive_attrition type.

agents/hub/e2e_automation.py

Lines 73–122 · Initiate, coordinate, and scan SLA

```

73     state["initiation"] = service.activate_case(state["case_id"])
74     return state
75
76
77 @traced_node("E2E -- coordinate stages")
78 def _coordinate(state: E2EState) -> E2EState:
79     # If the case wasn't found, skip the full pipeline and let _finalize report it.
80     if state["initiation"].get("error"):
81         state["pipeline"] = {"skipped": state["initiation"]["error"]}
82         return state
83     # supervisor.run_case() drives the entire HR -> manager -> IT -> finance -> assess pipeline.
84     state["pipeline"] = supervisor.run_case(
85         state["case_id"],
86         kt_text=state.get("kt_text"),
87         interview_text=state.get("interview_text"),
88         simulate_rejection=state["simulate_rejection"],
89     )
90     return state
91
92
93 @traced_node("E2E -- SLA breach check (this case only)")
94 def _sla_check(state: E2EState) -> E2EState:
95     case_id = state["case_id"]
96     if state["initiation"].get("error"):
97         state["sla"] = {"skipped": "case not found"}
98         return state
99
100     # Check ONLY this case's pending tasks — not the global scan (which would
101     # escalate unrelated cases as a side effect of running this one case).
102     tasks = (
103         db.table("exit_tasks").select("id, case_id, stage, title, status, due_date")
104         .eq("case_id", case_id).eq("status", "pending").execute().data or []
105     )
106     case_row = db.table("exit_cases").select("id, employee_name, department, hr_id, manager_id") \
107         .eq("id", case_id).single().execute().data or {}
108     cases = {case_row["id"]: case_row} if case_row else {}
109     profile_ids = [pid for pid in {case_row.get("hr_id"), case_row.get("manager_id")} if pid]
```

```

110     profiles = (
111         {p["id"]: p for p in db.table("profiles").select("id, full_name").in_("id", profile_ids).execute().data}
112         if profile_ids else {}
113     )
114     breaches = sla_escalation.find_breaches(tasks, cases, profiles, date.today())
115     # _escalate() sends emails for each breach and returns how many were sent.
116     escalated = sla_escalation._escalate({"breaches": breaches, "escalated": 0})["escalated"]
117     state["sla"] = {"breaches_found": len(breaches), "escalated": escalated}
118     return state
119
120
121 @traced_node("E2E -- finalize")
122 def _finalize(state: E2EState) -> E2EState:

```

Lines 73–77: Reuse the same activation function as the browser service.

Lines 80–91: Skip missing cases or call the supervisor with supplied state.

Lines 94–122: Scope the SLA scan to this case, resolve owner names, reuse finder/escalator, and store counts.

agents/hub/e2e_automation.py

Lines 125–177 · Finalize from persisted gates

```

125     if state["initiation"].get("error"):
126         state["outcome"] = {"status": "blocked", "reason": state["initiation"]["error"]}
127         return state
128     if state["simulate_rejection"]:
129         state["outcome"] = {"status": "blocked", "reason": "manager rejected KT plan -- escalated to HR"}
130         return state
131
132     # Read back the task rows that the pipeline actually wrote to decide the outcome.
133     # This prevents a false "completed" when a pipeline step partially failed.
134     compliance_row = db.table("exit_tasks").select("status, title").eq("case_id", case_id).eq("stage",
"compliance").execute().data
135     finance_row = db.table("exit_tasks").select("status, title").eq("case_id", case_id).eq("stage",
"finance").execute().data
136     compliance_done = bool(compliance_row) and compliance_row[0]["status"] == "done"
137     finance_done = bool(finance_row) and finance_row[0]["status"] == "done"
138
139     if compliance_done and finance_done:
140         # Both clearance gates passed — mark the case complete.
141         db.table("exit_cases").update({"status": "completed"}).eq("id", case_id).execute()
142         log_db("update", "exit_cases", rows=1, detail="status -> completed")
143         state["outcome"] = {"status": "completed"}
144     else:
145         # Name the specific blocking items so the caller can surface a useful message.
146         blocking = []

```

```

147         if not compliance_done:
148             blocking.append(compliance_row[0]["title"] if compliance_row else "compliance: not run")
149         if not finance_done:
150             blocking.append(finance_row[0]["title"] if finance_row else "finance: not run")
151         state["outcome"] = {"status": "blocked", "reason": "; ".join(blocking)}
152
153     db.table("agent_runs").insert(
154         {"case_id": case_id, "stage": "e2e_automation", "detail": f"final outcome: {state['outcome']}"}
155     ).execute()
156     log_db("insert", "agent_runs", rows=1, detail=str(state["outcome"]))
157     return state
158
159
160 # Four-node graph: initiate -> coordinate -> sla_check -> finalize.
161 _graph = StateGraph(E2EState)
162 _graph.add_node("initiate", _initiate)
163 _graph.add_node("coordinate", _coordinate)
164 _graph.add_node("sla_check", _sla_check)
165 _graph.add_node("finalize", _finalize)
166 _graph.set_entry_point("initiate")
167 _graph.add_edge("initiate", "coordinate")
168 _graph.add_edge("coordinate", "sla_check")
169 _graph.add_edge("sla_check", "finalize")
170 _graph.set_finish_point("finalize")
171 e2e_graph = _graph.compile()
172
173
174 @traced_node("E2E Automation -- run case start to finish")
175 def run(case_id: str, *, kt_text: str | None = None, interview_text: str | None = None,
176         simulate_rejection: bool = False) -> dict:
177     return e2e_graph.invoke({

```

Lines 125–135: Missing and simulated-rejection cases return blocked outcomes immediately.

Lines 137–154: Read the actual compliance/finance task rows; only two done rows allow completion.

Lines 156–170: Otherwise report precise blockers, audit the final outcome, and return state.

Lines 173–177: Begin wiring the four-node capstone graph.

9.9 Database definitions and gates

supabase/migrations/0001_schema.sql

Lines 49–89 · Case and task tables

```

49 create table if not exists exit_cases (
50     id                uuid primary key default gen_random_uuid(),
51     employee_id       text not null references profiles (employee_id),

```

```

52     employee_name    text not null,
53     email            text not null,
54     department       text not null,
55     role_title       text not null,
56     manager_id       uuid references profiles (id),
57     hr_id            uuid references profiles (id),
58     last_working_day date not null,
59     status           text not null default 'open' check (status in ('open', 'in_progress', 'completed')),
60     -- HR-only assessment columns below this line.
61     risk_level       text check (risk_level in ('low', 'medium', 'high')),
62     risk_score       numeric,
63     rehire_eligible  boolean,
64     created_at       timestamptz not null default now()
65 );
66
67 create index if not exists idx_exit_cases_employee_id on exit_cases (employee_id);
68 create index if not exists idx_exit_cases_manager_id  on exit_cases (manager_id);
69 create index if not exists idx_exit_cases_status      on exit_cases (status);
70
71 -- -----
72 -- exit_tasks - checklist items. `stage` routes a task to a dashboard
73 -- ('it' -> IT dashboard, 'finance' -> HR, etc). kt_event_id is written by
74 -- Phase 8 when a KT meeting is booked.
75 -- -----
76 create table if not exists exit_tasks (
77     id          uuid primary key default gen_random_uuid(),
78     case_id     uuid not null references exit_cases (id) on delete cascade,
79     stage       text not null,
80     title       text not null,
81     status      text not null default 'pending' check (status in ('pending', 'done')),
82     due_date    date,
83     kt_event_id text,
84     created_at  timestamptz not null default now()
85 );
86
87 create index if not exists idx_exit_tasks_case_id on exit_tasks (case_id);
88 create index if not exists idx_exit_tasks_stage  on exit_tasks (stage);
89

```

Lines 49–69: exit_cases stores identity/workflow fields and HR-only risk fields, with foreign keys to profiles.

Lines 76–86: exit_tasks stores stage, title, forward-only status values, due date, and optional calendar id.

Lines 88–89: Indexes support the dominant case/stage filters.

supabase/migrations/0001_schema.sql

Lines 125–138 · RAG table and vector index

```

125 create table if not exists exit_docs (
126     id          bigserial primary key,
127     source      text not null,      -- file the chunk came from, e.g. 'exit_policy.md'
128     section     text,              -- human label, e.g. '$4.2 Final settlement'
129     content     text not null,     -- the chunk text itself
130     embedding   vector(1536)       -- the chunk, as a vector
131 );
132
133 -- HNSW (Hierarchical Navigable Small World) is a vector index type that enables
134 -- fast approximate nearest-neighbour search without needing training data.
135 -- vector_cosine_ops means "use cosine similarity" (angle between vectors) as
136 -- the distance metric. This is what makes the /ask RAG search fast.
137 create index if not exists idx_exit_docs_embedding
138     on exit_docs using hnsw (embedding vector_cosine_ops);

```

Lines 125–131: Each exit_docs row is one source-labelled text chunk and one 1536-dimensional vector.

Lines 136–138: HNSW with cosine operators accelerates nearest-neighbour ordering.

supabase/migrations/0002_rls.sql

Lines 31–49 · Current-role helper and RLS enablement

```

31 create or replace function public.app_current_role()
32 returns text
33 language sql
34 stable
35 security definer
36 set search_path = ''
37 as $$
38     select p.role
39     from public.profiles p
40     where p.id = auth.uid();
41 $$;
42
43 comment on function public.app_current_role() is
44     'Returns profiles.role for the calling auth user. security definer + empty search_path so RLS policies can call it
45     without granting read access to profiles.';
46
47 revoke all on function public.app_current_role() from public;
48 grant execute on function public.app_current_role() to authenticated;
49 -- -----

```

Lines 31–42: A stable security-definer helper maps auth.uid() to profiles.role with a fixed search path.

Lines 46–49: RLS is enabled on the main tables before policies grant narrow access.

supabase/migrations/0013_finance_role.sql

Lines 15–26 · Finance schema extension

```

15     check (role in ('employee', 'hr', 'manager', 'it', 'finance'));
16
17 alter table public.exit_cases
18     add column if not exists finance_cleared boolean not null default false,
19     add column if not exists dues_note text;
20
21 -- -----
22 -- exit_tasks: finance needs every stage's status (hr/manager/it/finance) to
23 -- know what's still pending on a case -- same "checklist isn't sensitive"
24 -- reasoning as the existing hr/it/manager/employee policies.
25 -- -----
26 drop policy if exists exit_tasks_finance_select on public.exit_tasks;

```

Lines 15–17: Replace the original role constraint so finance becomes valid.

Lines 19–21: Add the clearance flag and dues note to exit_cases.

Lines 25–26: Begin finance task visibility policy.

supabase/migrations/0017_relieving_letter.sql

Lines 29–72 · Database-enforced final gate

```

29 create or replace function public.exit_case_cleared_for_relieving(p_case_id uuid)
30 returns boolean
31 language sql
32 security definer
33 set search_path = public
34 stable
35 as $$
36     select coalesce(
37         (select finance_cleared from public.exit_cases where id = p_case_id),
38         false
39     )
40     and not exists (
41         select 1 from public.exit_tasks
42         where case_id = p_case_id
43             and stage in ('hr', 'manager', 'it', 'finance')
44             and status <> 'done'
45     )
46     and (
47         select count(distinct stage) from public.exit_tasks
48         where case_id = p_case_id
49             and stage in ('hr', 'manager', 'it', 'finance')
50     ) = 4;
51 $$;

```

```

52
53 revoke all on function public.exit_case_cleared_for_relieving(uuid) from public;
54 grant execute on function public.exit_case_cleared_for_relieving(uuid) to authenticated;
55
56 grant update (relieving_letter_issued, issued_at, issued_by, status) on public.exit_cases to authenticated;
57
58 drop policy if exists exit_cases_hr_relieving_letter on public.exit_cases;
59 create policy exit_cases_hr_relieving_letter on public.exit_cases
60     for update to authenticated
61     using (
62         public.app_current_role() = 'hr'
63         and relieving_letter_issued = false
64     )
65     with check (
66         public.app_current_role() = 'hr'
67         and relieving_letter_issued = true
68         and issued_by = auth.uid()
69         and status = 'completed'
70         and public.exit_case_cleared_for_relieving(id)
71     );
72

```

Lines 29–49: The helper requires finance clearance, no unfinished hr/manager/it/finance task, and all four stages present.

Lines 51–54: Only authenticated callers may execute the helper.

Lines 56–74: HR update policy permits a one-way issue only when caller/issued_by/status/gate all match.

supabase/migrations/0030_finance_rpc_null_safe_authz.sql

Lines 34–69 · Finance authorization in RPCs

```

34 create or replace function public.finance_mark_dues_settled(p_case_id uuid, p_dues_note text default null)
35 returns void
36 language plpgsql
37 security definer
38 set search_path = ''
39 as $$
40 begin
41     -- NULL-safe: an unauthenticated caller (role NULL) is rejected here.
42     if public.app_current_role() is distinct from 'finance' then
43         raise exception 'not authorized';
44     end if;
45
46     update public.exit_cases
47         set finance_cleared = true,
48             finance_rejected = false,
49             dues_note = p_dues_note

```

```

50         where id = p_case_id;
51     end;
52 $$;
53
54 create or replace function public.finance_reject_dues(p_case_id uuid, p_reason text)
55 returns void
56 language plpgsql
57 security definer
58 set search_path = ''
59 as $$
60 begin
61     -- NULL-safe: an unauthenticated caller (role NULL) is rejected here.
62     if public.app_current_role() is distinct from 'finance' then
63         raise exception 'not authorized';
64     end if;
65
66     if p_reason is null or btrim(p_reason) = '' then
67         raise exception 'reason required';
68     end if;
69

```

Lines 34–50: Settlement is security-definer but explicitly checks `app_current_role() IS DISTINCT FROM finance` before updating.

Lines 54–69: Rejection repeats the same role check and requires a nonblank reason before clearing the flag.

9.10 CLI wrapper

agents/run_case.py

Lines 35–63 · Parse files/flags and run supervisor

```

35 )
36
37
38 def _read(path: str) -> str:
39     with open(path, encoding="utf-8") as f:
40         return f.read()
41
42
43 if __name__ == "__main__":
44     if len(sys.argv) < 2:
45         print(__doc__)
46         sys.exit(1)
47
48     case_id = sys.argv[1]
49     # sys.argv[2:] contains all flags after the case_id, e.g. ["--kt-text", "path.txt", "--reject"].

```

```

50     args = sys.argv[2:]
51     # args.index("--kt-text") finds where the flag appears, then +1 gets the file path right after it.
52     # If the flag isn't in args, fall back to the hardcoded demo text so KT-review still runs.
53     kt_text = _read(args[args.index("--kt-text") + 1]) if "--kt-text" in args else DEFAULT_KT_TEXT
54     interview_text = (
55         _read(args[args.index("--interview-text") + 1]) if "--interview-text" in args else DEFAULT_INTERVIEW_TEXT
56     )
57
58     # time.perf_counter() is the highest-resolution clock Python has – good for measuring
59     # how long the full pipeline takes (wall time, in seconds as a float).
60     start = time.perf_counter()
61     result = _run_case(case_id, kt_text=kt_text, interview_text=interview_text, simulate_rejection="--reject" in args)
62     elapsed = time.perf_counter() - start
63     print(f"\ncase {case_id} finished in {elapsed:.2f}s -- log: {result['log']}")

```

Lines 35–37: `_read` loads optional text files as UTF-8.

Lines 40–48: Require `case_id` and split remaining CLI flags.

Lines 49–56: Use provided files or built-in demo texts.

Lines 59–63: Time the supervisor call, pass rejection flag, and print the returned log.

End state: The live browser completes a case only through the HR relieving-letter update and its PostgreSQL gate. The CLI supervisor stops after risk assessment; the separate `e2e_automation` runner can set completed after persisted compliance and finance rows are both done.